International Journal of Production Research

ISSN: 0020-7543 (Print) 1366-588X (Online) Journal homepage: www.tandfonline.com/journals/tprs20

# Mathematical modelling and a meta-heuristic for flexible job shop scheduling

V. Roshanaei, Ahmed Azab & H. ElMaraghy

**To cite this article:** V. Roshanaei, Ahmed Azab & H. ElMaraghy (2013) Mathematical modelling and a meta-heuristic for flexible job shop scheduling, International Journal of Production Research, 51:20, 6247-6274, DOI: 10.1080/00207543.2013.827806

**To link to this article:** https://doi.org/10.1080/00207543.2013.827806

Published online: 15 Aug 2013.

*International Journal of Production Research*, 2013
Vol. 51, No. 20, 6247–6274, http://dx.doi.org/10.1080/00207543.2013.827806

Taylor & Francis
Taylor & Francis Group

# Mathematical modelling and a meta-heuristic for flexible job shop scheduling

V. Roshanaei, Ahmed Azab* and H. ElMaraghy

*Intelligent Manufacturing System (IMS) Centre, University of Windsor, Windsor, Canada*

(*Received 23 July 2012; final version received 16 July 2013*)

This study develops new solution methodologies for the flexible job shop scheduling problem (F-JSSP). As a first step towards dealing with this complex problem, mathematical modellings have been used; two novel effective position- and sequence-based mixed integer linear programming (MILP) models have been developed to fully characterise operations of the shop floor. The developed MILP models are capable of solving both partially and totally F-JSSPs. Size complexities, solution effectiveness and computational efficiencies of the developed MILPs are numerically explored and comprehensively compared vis-à-vis the makespan optimisation criterion. The acquired results demonstrate that the proposed MILPs, by virtue of its structural efficiencies, outperform the state-of-the-art MILPs in literature. The F-JSSP is strongly NP-hard; hence, it renders even the developed enhanced MILPs inefficient in generating schedules with the desired quality for industrial scale problems. Thus, a meta-heuristic that is a hybrid of Artificial Immune and Simulated Annealing (AISA) Algorithms has been proposed and developed for larger instances of the F-JSSP. Optimality gap is measured through comparison of AISA's suboptimal solutions with its MILP exact optimal counterparts obtained for small- to medium-size benchmarks of F-JSSP. The AISA's results were examined further by comparing them with seven of the best-performing meta-heuristics applied to the same benchmark. The performed comparative analysis demonstrated the superiority of the developed AISA algorithm. An industrial problem in a mould- and die-making shop was used for verification.

**Keywords:** scheduling; flexible job shop; mixed integer linear programming; hybrid artificial immune algorithms; simulated annealing; size complexity; optimality gap

## 1. Introduction

Flexible job shop scheduling is challenging due to the expanding machine tools capabilities and increased products variety. Full utilisation of added capabilities, versatilities and increased flexibilities in job shops makes scheduling these resources extremely challenging. The growing competition in international markets has generated demand for faster and more versatile machine tools, while preserving or improving the final product quality. The emergence of multi-purpose machine tools provided manufacturers with competitive capabilities. However, managing change in products and markets requires adaptation at two levels – physical enablers on one hand by making available flexible and reconfigurable machine tools and systems and logical enablers on the other by developing the required adaptable manufacturing functions such as re-configurable controls, process planning, production planning and scheduling (PPS) (Wiendahl et al. 2007). PPS map the production load of a factory to its capabilities and capacities in different time horizons and with different levels of detail. The two levels of PPS are coupled, since planning sets the goals as well as the resource and temporal constraints for scheduling (Vancza, Kis, and Kovacs 2004). Scheduling is responsible for unfolding a plan into detailed resource assignments and sequences. Production scheduling in real industrial environment presents additional challenges of size, which is normally larger than the capabilities of most existing algorithms and typical benchmarks in the literature.

### 1.1 *Scope and context*

The manufacturing setting studied in this study is a generalised variant of the classical job shop (JSSP) production systems known as F-JSSP; F-JSSP extends the job shop production systems by assuming that each machine is capable

*Corresponding author. Email: azab@uwindsor.ca

of offering more than one operation (Yazdani, Amiri, and Zandieh 2010). According to (Kacem, Hammadi, and Borne 2002), flexibility in job shop (F-JSSP) may be either partial or total – referred to in this work as either partially flexible job shop scheduling problem (PF-JSSP) or totally flexible job shop scheduling problem (TF-JSSP). The TF-JSSP is a special case of PF-JSSP, wherein PF-JSSP the preferences or feasibility of using some machines for certain operations arise; JSSP is a limit case where by definition no flexibility in operation assignment is allowed at all. In PF-JSSP, there exist certain number of multi-purpose machines distributed throughout the facility, the versatility and flexibility of which are not identical. This feature enables a certain job to be processed by at least one machine out of the available feasible machines.

As of TF-JSSP, this would be another limit case of PF-JSSP, where all machines are sufficiently flexible to perform all jobs. The focus of this research paper is on PF-JSSP. The routing flexibility of jobs permits dynamic re-assignment of jobs to other available machines in case of facing unforeseen events on the shop floor like machine breakdowns, order cancellation, new arrivals (Kesen and Gungor 2012). Alternate routing is useful when capacity problems arise. In the PF-JSSP, there are $m$ machines in the system and $n$ jobs to be processed. Each job $j$ requires $n_j$ operations to be performed; precedence constraints exist between operations of a job and are dealt within this work by following natural logical order of processing where these precedence restraints are satisfied. Each operation $O_{j,l}$, where $l$ is operation index, can be processed on a number of non-identical or identical machines and the processing time differs based on the machine characteristics. This addresses the existence of multiple routings for some jobs. An alternate routing could be used if a machine tool is temporarily overloaded while another feasible one is available. Therefore, based on (Brandimarte 1993) in F-JSSP, two distinct decisions have to be made:

- *Routing* (*assignment problem*): In this stage, operations are assigned to their respective feasible machines. For TF-JSSP as explained, any arbitrary assignment of operations to available machines is feasible since all machines in the shop should possess same operational capabilities. But, for the PF-JSSP, attention should be paid to the feasibility of machine assignments as machines do not possess identical machining capability.
- *Sequencing*: Once assignment decisions were made, the sequencing decision-making process is triggered. It goes without saying that sequencing decision is made for those operations whose processing route entails sharing the same machine; that is, those operations that are not assigned to the same machine are not considered for sequencing with respect to each other on that particular machine. Therefore, commonality in machine assignment calls for sequencing decisions.

### 1.2 *Assumptions*

- No process plan flexibility is considered.
- Jobs are independent and no priorities are assigned.
- Pre-emption or cancellation of jobs is not considered.
- Each machine can only process one job at a time.
- Each job can be processed by only one machine at a time.
- No processor blocking occurs due to finite in-process storage.
- Processing times are deterministic and include other elements of set-up, operations, transportation and inspection.
- All jobs are inspected a priori; that is, no defective job is considered.
- The orders volume is known a priori and jobs are simultaneously available at time zero.
- Machine breakdowns are not considered.

### 1.3 *Representation of F-JSSP by standard triplet*

In order to facilitate the solution, the PF-JSSP is transformed into the TF-JSSP by adding 'infinite processing times' referred to as big $M$ to the incapable machines. Scheduling problems are usually represented by a standard triplet ($\alpha \mid \beta \mid \gamma$). According to Pinedo (2002), the TF-JSSP can be denoted by $FJc|C_{\max}$; PF-JSSP is represented as follows: $FJc|M_i\ |C_{\max}$ *since machine feasibility becomes relevant*. The first symbol indicates the type of shop that is F-JSSP, while the second symbol indicates machine eligibility issue. Finally, the third symbol denotes the objective function that is make-span. The problem studied in this study encompasses both TF-JSSP and PF-JSSP.

### 1.4 *Size complexity of F-JSSP*

In TF-JSSP, a certain number of multi-purpose machines ($m$) on the shop floor exist. For any of these multi-purpose machine tools, as many feasible schedules as factorial of number of jobs ($n!$) can be generated. This is equivalent to the complexity of the single machine scheduling. Since there is more than one machine on the shop floor, jobs have to be considered for sequencing on other flexible machines as well. This issue begets tremendous complexity for F-JSSP and makes its complexity rise to $(n!)^m$. The classical JSSP has proven to be strongly NP-hard by (Garey, Johnson, and Sethi 1976). Therefore, the F-JSSP is also NP-hard.

### 1.5 *General approaches to F-JSSP*

Due to the tremendous complexity of the F-JSSP, meta-heuristics are utilised. For tackling F-JSSP, two universally established frameworks have been proposed:

- *Hierarchical*: For this approach, assignment of operations to their respective machines is initially made and well after that sequencing procedure starts. In effect, hierarchical approach reduces the complexity of F-JSSP by decomposing it into two sub-problems: feasible operation-machine assignment and sequencing (Bagheri et al. 2010).
- *Integrated*: In this approach, both assigning and sequencing decisions are made concurrently.

Bagheri et al. (2010) claimed that integrated approaches are superior to hierarchical approaches; accuracy of such a claim is investigated in this study.

### 1.6 *Proposed methodologies for the F-JSSP*

As was mentioned in the abstract, this study utilises two widely known methodologies to fulfil the scheduling requirements of F-JSSP:

- *Mathematical modelling*: Two effective mathematical formulations are developed in the form of Mixed Integer Linear Programming (MILP) for both TF-JSSP and PF-JSSP.
- *Hybrid meta-heuristic*: A hybrid meta-heuristic algorithm resulted from hybridisation of Artificial Immune Algorithm and Simulated Annealing (AISA) is presented. The hybrid technique follows the *hierarchical* approach to solve the problem.

### 1.7 *Objective function:*

Several objective functions were considered. Among all objective functions found in the literature, the make-span ($C_{\max}$) or maximum completion times of operations were selected. The significance of the selected objective function is addressed in Section 3.2.1.

## 2. Literature review

Two classes of solution methodologies are being employed in this work, namely mathematical programming and meta-heuristics for tackling F-JSSP; thus, the literature survey has also been divided into two sub-sections: (a) review of mathematical models that have been proposed for F-JSSP with make-span optimisation criterion and (b) review of meta-heuristics and role of hybridisation between the techniques.

### 2.1 *Review of mathematical programming formulations*

Mathematical modelling, a solution technique for the production scheduling problem, was established as a viable approach after the publication of the seminal work of Wagner (1959). Afterwards, several mathematical formulations were proposed for the scheduling of different shop types. A recent literature survey by Ozguven, Yavuz, and Ozbakir (2012) has compiled all the relevant mathematical models for F-JSSP. According to Ozguven, Yavuz, and Ozbakir

Table 1. Related MILP models for F-JSSP with make-span optimisation criterion.

| References | Names | Modelling paradigm | Objective function |
|---|---|---|---|
| Fattahi, Mehrabad, and Jolai (2007) | MILP-1 | Position-based | Make-span |
| Ozguven, Ozbakir, and Yavuz (2010) | MILP-2 | Sequence-based | Make-span |
| Roshanaei, ElMaraghy, and Azab (2012) | MILP-3 | Sequence-based | Make-span |

(2012), only Fattahi, Mehrabad, and Jolai (2007) has formulated F-JSSP with make-span optimisation criterion based on Wagner's definition of binary variables (BVs). Their model is henceforth referred to as MILP-1 in this work. There are also two other MILPs in the literature that used make-span optimisation criterion, (Ozguven, Ozbakir, and Yavuz 2010) and (Roshanaei, ElMaraghy, and Azab 2012). The latter has been formulated based on Manne's definition of BVs and hereinafter are referred to as MILP-2 and MILP-3, respectively (Table 1).

In this study, a new MILP formulation (MILP-4) has been developed which absolutely improves on MILP-1. Both MILP-1 and MILP-4 follow Wagner's definition of binary integer variables and are henceforth called position-based MILPs. Also, a completely enhanced MILP formulation based on Manne's definition of binary integer variable (MILP-5) is developed to improve on both MILP-2 and MILP-3.

### 2.2 *Review of hybrid meta-heuristics*

To solve F-JSSP as a whole, meta-heuristics and other solution techniques have been exploited. Influential and state-of-the-art solution techniques are reviewed in this sub-section. Brucker and Schlie (1990) were the first to tackle F-JSSP. They developed a polynomial algorithm for solving F-JSSP with two jobs. Brandimarte (1993) utilised a hierarchical approach for F-JSSP and solved the routing sub-problem using some existing dispatching rules and sequencing sub-problem using a Tabu Search (TS) algorithm. Hurink, Jurisch, and Thole (1994) and Barnes and Chambers (1996) developed TS algorithms based on hierarchical and integrated approach to solve F-JSSP, respectively. Dauzere-Peres and Paulli (1997) proposed a new neighbourhood structure for the problem and proposed a TS algorithm based on the integrated approach. Mastrolilli and Gambardella (2000) enhanced (Dauzere-Peres and Paulli 1997) TS techniques and proposed two neighbourhood functions. Chen, Ihlow, and Lehmann (1999) developed a genetic algorithm (GA) approach for F-JSSP. Their encoding scheme included both the routing policy and the sequence of operations on each machine. Kacem, Hammadi, and Borne (2002) proposed a GA controlled by the assigned model that was generated by the approach of localisation, and they used it to solve mono-objective and multi-objective F-JSSP. Xia and Wu (2005) proposed a hierarchical approach for solving multi-objective F-JSSP. Particle swarm optimisation was used to assign operations to machines and simulated annealing (SA) was then employed to schedule operations on each machine. Fattahi, Mehrabad, and Jolai (2007) developed the first position-based mathematical model with make-span optimisation; they proposed six new integrated and hierarchical meta-heuristics-based SA and TS for solving F-JSSP. Pezzella, Morganti, and Ciaschetti (2008) developed an enhanced GA for F-JSSP based on an integrated approach. They utilised a mix of strategies for generating the initial population, selecting individuals for reproduction, and reproducing new individuals were presented. Gao et al. (2008) studied the F-JSSP with three objectives: min make-span, min maximal machine workload and min total workload. They developed a hybrid genetic algorithm. Yazdani, Amiri, and Zandieh (2010) developed a parallel variable neighbourhood search algorithm for solving F-JSSP. They showed that parallelisation in the presented optimisation method increased the diversification and the exploration in the search space. In their method, neighbourhood structures related to sequencing and assignment problems are employed to generate a neighbouring solution. They performed comprehensive evaluations among different meta-heuristics and showed their proposed algorithms works better. Bagheri et al. (2010) proposed a new artificial immune algorithm (AIA) for F-JSSP. They applied their algorithm to a data set proposed by (Fattahi, Mehrabad, and Jolai 2007) and also several other data sets and proved the efficiency of their algorithm. They showed their algorithm completely outperforms those six hybrid meta-heuristics proposed by (Fattahi, Mehrabad, and Jolai 2007). Zhang, Gao, and Shi (2011) developed an effective GA to minimise make-span, where both global and local selection was utilised for initialisation. They demonstrated that their algorithms outperform other algorithms on certain benchmarks. Al-Hinai and ElMekkawy (2011) developed a new hybrid GA for F-JSSP. They integrated GA with and a local search method, while developing a dedicated algorithm for population initialisation. They applied their proposed hybrid GA to wide variety of instances of F-JSSP and showed that their algorithm produced better results than previously developed algorithms.

In this section, pioneering and state-of-the-art algorithms for F-JSSP were reviewed. All of the above-cited algorithms have been applied to different data sets. The following is a summary of benchmark problems found in the literature to solve F-JSSP:

(1) *Kacem data*: This is a set of three problems (problem $8*8$, problem $10*10$ and problem $15*10$) that are all taken from (Kacem, Hammadi, and Borne 2002).
(2) *BR-data*: This is composed of ten test problems from (Brandimarte 1993). The number of jobs, machines and operations ranges from 10 to 20, 4 to 15 and 5 to 15, respectively. All in all, the number of operations for all jobs ranges from 55 to 240.
(3) *BC-data*: This is a set of 21 problems (Barnes and Chambers 1996). The number of jobs, machines and operations ranges from 10 to 15, 11 to 18 and 10 to 15, respectively; finally, number of operations for all jobs ranges from 100 to 225.
(4) *DP-data*: This consists of 18 problems from (Dauzere-Peres and Paulli 1997). The number of jobs ranges from 10 to 20; number of machines ranges from 5 to 10; number of operations for each job ranges from 15 to 25; finally, number of operations for all jobs ranges from 196 to 387.
(5) *HU-data*: This is a set of 129 problems from (Hurink, Jurisch, and Thole 1994). It consists of three problems (mt06, mt10 and mt20) drawn from Fisher and Thompson (1963) and 40 (la01–la40) from Lawrence (1984). The number of jobs ranges from 6 to 30; the number of machines ranges from 5 to 15; the number of operations for each job ranges from 5 to 15; finally, the number of operations for all jobs ranges from 36 to 300.
(6) *F-data*: This consists of 20 test problems (Fattahi, Mehrabad, and Jolai 2007). These test problems are divided into two categories: small-size flexible job-shop scheduling problems (SFJS1:10) and medium and large size flexible job-shop scheduling problems (MFJS1:10). The number of jobs ranges from 2 to 12; the number of machines ranges from 2 to 8, the number of operations for each job ranges from 2 to 4; finally, the number of operations for all jobs ranges from 4 to 48.

In order to evaluate the effectiveness of meta-heuristics comparisons and contrasting against other meta-heuristics to solve same benchmark problems have been carried out. However, in cases where the optimal solution of a problem is known beforehand, optimality gap measurements of meta-heuristics are the best and sole criterion for measuring their absolute strength. To measure the optimality gap of meta-heuristics, they have to be applied to data sets for which optimal or near-optimal solutions are known a priori. The F-data set have been used (small-to-medium instances) with make-span optimisation criterion proposed by (Fattahi, Mehrabad, and Jolai 2007) to evaluate the performances of the MILPs. State-of-the-art meta-heuristics from the literature that have used this benchmark are being used and compared against the developed AISA heuristic. It is certain that comparative performance evaluations among meta-heuristics provide relative information regarding their strength; their optimality deviation is never measured though. Optimality gap (Absolute Performance Deviation) of the relevant meta-heuristics is measured in solving the 20 instances of F-data set – see Table 2.

According to literature, Fattahi, Mehrabad, and Jolai (2007) have developed six hybrid meta-heuristics based on SA and TS algorithms. Some of these algorithms follow an integrated approach, while some others hierarchical approach for solving F-JSSP with make-span as the optimisation criterion. Bagheri et al. (2010) also used original AIA for solving F-JSSP and make-span as their objective function for F-JSSP. The AIA follows an integrated approach. In total, these seven meta-heuristics (Fattahi, Mehrabad, and Jolai 2007; Bagheri et al. 2010) that have been developed for F-JSSP and applied to the F-data set have been used for comparison against the developed AISA approach. By quantification of algorithms' error, quality of the different approaches applied to F-JSSP is decided.

### 2.3 *Statement of contributions*

Generally speaking, this study theoretically and practically contributes to the solution methodologies thus far presented in the field of production scheduling and F-JSSP in particular. Contributions are summarised as follows:

Table 2. Literature survey for related meta-heuristic algorithms using F-data set.

| Author | Type | Problem addressed | Objectives |
|---|---|---|---|
| Fattahi, Mehrabad, and Jolai (2007) | Six TS-SA | F-JSSP | Make-span |
| Bagheri et al. (2010) | AIA | F-JSSP | Make-span |

- Development of two new position- and sequence-based MILPs for PF-JSSP and TF-JSSP, which structurally and computationally outperform their counterparts in the literature (MILP-1, MILP-2 and MILP-3). These models are proposed to capture all aspects of shop floor operations and solve the F-JSSP to medium-size instances of the problem.
- Owing to the tremendous complexity of F-JSSP, which leads to its strong NP-hardness, MILPs are rendered inefficient in solving larger instances of the problem. Therefore, the AISA hybrid solution methodology is proposed, which outperforms seven of the best-performing meta-heuristics in the literature.
- Last but not least, the proposed AISA is applied to a real industrial problem in a mould- and die-making shop for verification purposes.

## 3. Proposed mathematical models

The speed with which integer programming problems can be solved depends upon the number of BVs, constraints and continuous variables in the problem (French 1982), where the most deciding factor is the number of BVs. If two formulations have the same number of BVs, Liao and You (1992) and Wilson (1989) showed that the next influential element is the number of constraints.

As has been explained, mathematical programming as a solution technique for production scheduling problems found its significance after the seminal work of Wagner (1959). Soon after Wagner proposed his first model which was position-based, Bowman (1959) propounded his time-based model for the same problem. Shortly after, Manne (1960) presented his efficient sequence-based mathematical modelling paradigm. Researchers have ever since been basing their work on these ground-breaking modelling schemes and extending them in different ways to solve job shop scheduling problems. The proposed models in this study are position- and sequence-based, since they have proven to be computationally and mathematically efficient (Pan 1997). The number of BVs that the time-based modelling paradigm generates directly depends on the arbitrary determination of the time-slots for each machine. Let us assume the planning horizon is daily and we have two working shifts in a day with 16 h in total. This time span (16 h) can be partitioned further into an arbitrary number of time-slots. Each time-slot is equivalent to a BV in Bowman model. For example, in extreme case, a manufacturing company may define per minute time-slots on its machines which is equivalent to the generation of 960 ($16 \times 60$) BVs per day. If the planning horizon is broken down to discrete time-slots of 5 min, we will have 180 BVs. In the best case and based on hourly time-slots, we will have 16 BVs per machine per day. Therefore, the definition of T (time-slot) is completely arbitrary and can vary based on the problem specifics and user preference. The rationale behind employing the position-based and the sequence-based modelling paradigms is that the number of BVs that are generated is based on the number of jobs (fixed number) on each machine and the eligibility of that machine to process those jobs. Therefore, the number of BVs remains constant for position-and-sequence-based models; furthermore, this makes the comparison possible between the two models, whereas in the time-based formulations, the number of BVs is variable. Generally speaking, time-based integer formulations have proven to be computationally burdensome and solution-wise inferior to position- and sequence-based modelling paradigms (Pan 1997).

### 3.1 *Proposed position-based MILP model (MILP-4)*

Developing mathematical models for scheduling problems entails defining appropriate constraint sets to represent technical precedence among operations of a job; F-JSSP requires other constraint sets to ensure feasible assignment of operations to eligible machines.

*Parameters*

| | |
|---|---|
| $j$ | subscript for jobs where $1 \leqslant j \leqslant n$ |
| $i$ | subscript for machines where $1 \leqslant i \leqslant m$ |
| $l$ | subscript for operations of job $j$ where $1 \leqslant l \leqslant n_j$ |
| $e_{j,l,i}$ | eligibility parameter that takes value 1 if machine $i$ is able to process operation $O_{j,l}$ and 0 otherwise |
| $f$ | subscript for positions of machine $i$ where $1 \leqslant f \leqslant f_i$ ($f_i = \sum_j \sum_l e_{j,l,i}$) |
| $M$ | a large positive number |

*Decision variables*

| | |
|---|---|
| $X_{j,l,i,f}$ | binary decision variable taking value 1 if $l$th operation of job $j$ is processed on the $f$th position of machine $i$ |
| $C_{j,l}$ | continuous decision variable for completion time of operation $O_{j,l}$ |
| $F_{i,f}$ | continuous decision variable for finishing time of each processing position $f$ on machine $i$ |

3.1.1 *Objective function*

$$\text{Min } C_{\max} \tag{1}$$

The purpose of the proposed MILP is to find optimal or feasible schedules for F-JSSP. To this end, an appropriate objective function has been selected and employed. Various objective functions were explored and eventually maximum completion times of operations commonly referred to as make-span ($C_{\max}$) was selected as an optimisation criterion. The rationale behind utilising make-span as optimisation criterion is that make-span incorporates machine utilisation through reduction in idle time on all machines and to large extent ensures even workload distribution among work centres (Pan 1997).

3.1.2 *Assignment constraint sets*

*3.1.2.1 Constraints for assigning and sequencing of operations to available processing positions.*

$$\sum_{i}^{m} \sum_{f}^{f_i} X_{j,l,i,f} = 1 \quad \forall j, l \tag{2}$$

This constraint set ensures that each operation is uniquely assigned to one processing position of a machine among all available machines. This constraint set fulfils two decision levels that are being considered simultaneously, assignment and sequencing, by virtue of the subscripts $i$ and $f$ in $X_{j,l,i,f}$.

*3.1.2.2 Machine utilisation constraints.*

$$\sum_{j}^{n} \sum_{l}^{n_j} X_{j,l,i,f} \leqslant 1 \quad \forall i, f \tag{3}$$

Constraint set (3) simply states the fact that *not* all existing machine capabilities in the shop floor are exploited. In other words, not all processing positions on machine tools remain allocated for each scheduling horizon just because number of processing positions far exceed number of operations.

*3.1.2.3 Machine eligibility constraints.*

$$\sum_{f}^{f_i} X_{j,l,i,f} \leq e_{j,l,i} \quad \forall j, l, i \tag{4}$$

In order for any operation to be processed on any machine that machine should possess the capability to process the operation. Constraint sets (2), (3) and (4) altogether ensure feasible assignments of operations to their respective and eligible machines.

3.1.3 *Sequencing constraint sets*

*3.1.3.1 Logical precedence constraints among operations of a job.*

$$C_{j,l} \geqslant C_{j,l-1} + \sum_{i}^{m} \sum_{f}^{f_i} X_{j,l,i,f} \cdot p_{j,l,i} \quad \forall j, l \tag{5}$$

Constraint set (5) reveals the fact that a job cannot be processed on two machines simultaneously. Technically speaking, the succeeding operation of a job can start once the preceding operation of that job has been completed on any of the flexible machines in the shop floor.

*3.1.3.2 Machine non-interference constraints.*

$$F_{i,f} \geqslant F_{i,f-1} + \sum_{j=1}^{n} \sum_{l=1}^{n_j} X_{j,l,i,f} \cdot p_{j,l,i} \quad \forall_{i,f} \tag{6}$$

By virtue of constraint (6), processing positions are related to each other. Processing positions on each machine become available as soon as the assigned operation to its preceding processing position has been completed its operational requirement.

*3.1.3.3 Constraints for relating finishing times of processing positions to completion times of operations.*

$$F_{i,f} \leqslant C_{j,l} + M(1 - X_{j,l,i,f}) \quad \forall_{j,l,i,f} \tag{7}$$

$$F_{i,f} \geqslant C_{j,l} - M(1 - X_{j,l,i,k}) \quad \forall_{j,l,i,f} \tag{8}$$

In order for a job to start its succeeding operation, its preceding operation has to be completed (constraint (5)). Also, the subsequent processing position has to be available (constraint (6)).

Jobs and machines face three possible states while finishing their current operation:

- The job can immediately join the already assigned machine without any waiting time.
- The job has to wait until its already assigned machine becomes available.
- Machine starves until the next job is loaded on it (machine idle time or starvation).

The above three states are mutually exclusive. Constraints sets (7) and (8) concurrently imply the fact that the earliest time a job can start its succeeding operation is when both the job and the machine are available.

*3.1.3.4 Constraints for capturing the value of objective function.*

$$C_{\max} \geqslant C_{j,n_j} \quad \forall_j \tag{9}$$

Constraint set (9) captures the value of make-span by considering the completion times of the last operation of all jobs.

*3.1.3.5 Constraints demonstrating the nature of decision variables.*

$$C_{j,l}, F_{i,f} \geqslant 0 \quad \forall_{j,l,i,f} \tag{10}$$

$$X_{j,l,i,f} \in \{0, 1\} \quad \forall_{j,l,i,f} \tag{11}$$

Constraint set (10) enforces that our continuous decision variables take positive values. Constraint set (11) shows the binary nature of the assignment and sequencing decision variable.

### 3.2 *Enhanced hierarchical sequence-based MILP model (MILP-5)*

*Parameters*

| | |
|---|---|
| $j, h$ | subscript for jobs where $1 \leqslant j, h \leqslant n$ |
| $i$ | subscript for machines where $1 \leqslant i \leqslant m$ |
| $l, z$ | subscript for operations of job where $1 \leqslant l, z \leqslant n_j$ |
| $M$ | a large positive number |
| $R_{j,l}$ | the set including machines eligible to process $O_{j,l}$ |
| $p_{j,l,i}$ | processing time of $l$th operation of job $j$ on machine $i$ |

*Decision variables*

| | |
|---|---|
| $X_{j,l,h,z}$ | BV taking value 1 if $O_{j,l}$ is processed after $O_{h,z}$ on machine $i$; and 0 otherwise $j<n, h>j$ |
| $Y_{j,l,i}$ | BV taking value 1 if $O_{j,l}$ is processed on machine $i$; and 0 otherwise |
| $S_{j,l,i}$ | continuous variable for the starting time of $O_{j,l}$ on machine $i$ |

*MILP-5*

This MILP model has been formulated based on Manne's definition of BVs.

3.2.1 *Objective function*

$$\text{Min } C_{\max} \tag{12}$$

3.2.2 *Assignment constraint sets*

$$\sum_{i=1}^{m} Y_{j,l,i} = 1 \quad \forall_{j,l} \tag{13}$$

Constraint set (12) ensures that all operations are investigated for processing on all available machines and are imperatively assigned to one of them.

$$S_{j,l,i} \leqslant M(Y_{j,l,i}) \quad \forall_{j,l,i} \tag{14}$$

Constraint set (13) demonstrates that starting times of operations take value if and only if that operation is assigned to that machine and 0 otherwise.

3.2.3 *Sequencing constraint sets*

*3.2.3.1 Logical precedence constraints among operations of a job.*

$$\sum_{i=1}^{m} S_{j,l+1,i} \geqslant \sum_{i=1}^{m} S_{j,l,i} + \sum_{i=1}^{m} Y_{j,l,i} \cdot p_{j,l,i} \quad \forall_{j,l<n_j} \tag{15}$$

Constraint set (14) represents the logical/natural precedence constraint among the operations of a job. It simply means that so long as the previous operation of a job has not been completed on any of the machines in the shop floor, the succeeding operation is not processed.

*3.2.3.2 Machine non-interference constraints and precedence constraints among operations of different jobs.*

$$S_{j,l,i} \geqslant S_{h,z,i} + p_{h,z,i} - M(3 - X_{j,l,h,z} - Y_{j,l,i} - Y_{h,z,i}) \quad \forall_{j<n,l;h>j,z;i\{R_{j,l} \cap R_{h,z}\}} \tag{16}$$

$$S_{h,z,i} \geqslant S_{j,l,i} + p_{j,l,i} - M(X_{j,l,h,z} + 2 - Y_{j,l,i} - Y_{h,z,i}) \quad \forall_{j<n,l;h>j,z;i\{R_{j,l} \cap R_{h,z}\}} \tag{17}$$

Constraints sets (15) and (16) are either-or constraints; they simultaneously, ensure the followings: (1) an operation cannot overlap with both predecessor and the successor operations and (2) satisfaction of non-interference constraints (precedence constraint among operations of different jobs); that is, for the operations of different jobs that are eligible to be processed on the same machine. In other words, if two operations of two different jobs do not share the same subset of machines, consideration of the operational precedence between them is not take place. Hence, two operations $O_{j,l}$ and $O_{h,z}$ can only be sequenced when both their binary integer variables assignment $Y_{j,l,i}$ and $Y_{h,z,i}$ take value of 1; otherwise, they bear no relationship with one another on machine $i$. Once machine $i$ was established as eligible machine to process $O_{j,l}$ and $O_{h,z}$ ($Y_{j,l,i} = 1$ and $Y_{h,z,i} = 1$), the precedence relationship between these two operations should be

decided by the sequencing binary decision variable which is $X_{j,l,h,z}$. If $O_{j,l}$ is processed after $O_{h,z}$ on machine $i$, the sequencing binary decision variable takes value 1, and therefore, constraint set (15) becomes active. Otherwise, this constraint shows that $C_{h,z}$ is just greater than a large negative number, which is naturally true. The sequencing BV ($X_{j,l,h,z}$) takes value 0 if $O_{j,l}$ is processed before $O_{h,z}$. Since two operations in sequencing decision have no more than two states with respect to each other (predecessor or successor), if $O_{j,l}$ does not succeed $O_{h,z}$, it has to precede it, in which case constraint set (16) becomes active and constraint set (15) becomes an evident inequality equation.

*3.2.3.3 Constraints for capturing the value of objective function.*

$$C_{\max} \geq \sum_{i=1}^{m} S_{j,n_j,i} + \sum_{i=1}^{m} Y_{j,n_j,i} \cdot p_{j,n_j,i} \ \forall_j \tag{18}$$

Constraint set (17) keeps track of make-span and compute it.

*3.2.3.4 Constraints demonstrating the nature of decision variables.*

$$S_{j,l,i} \geq 0 \quad \forall_{j,l,i} \tag{19}$$

$$X_{j,l,h,z}, Y_{j,l,i} \in \{0,1\} \quad \forall_{j,l,h,z}, \ \forall_{j,l,i} \tag{20}$$

Constraint set (3)–(18) shows the non-negativity nature of the MILP'S continuous variables. Constraints set (3)–(19) demonstrate the binary nature of decision variables.

## 4. Proposed hybrid meta-heuristic for the F-JSSP

The proposed meta-heuristic is a hybrid of AISA.

### 4.1 *Background and mechanics of proposed algorithm*

AIAs are adaptive computational systems that simulate the behaviour of the immune system of natural living organisms, where the body recognises foreign substances known as antigens and generates a set of antibodies to exterminate them. AIAs, by virtue of *affinity values* (*inverse of make-span values*), discern the *antibodies* (*feasible schedules*) that demonstrate more potential in exterminating *antigens* (*minimisation of PF-JSSP*) so as to further proliferate their respective variations in next generations of antibodies. Therefore, the effectiveness of each antibody is measured by its affinity value. In order for the AISA to be better comprehended, a one-by-one correspondence is established to relate the elements of the F-JSSP and the AISA. The correspondence is as follows:

- Antigens: antigen is the F-JSSP that is to be solved.
- Antibodies: antibodies are feasible schedules.
- Affinity values: are the inverses of objective function values. The lower the make-span is; the higher is the affinity value.

Cloning selection algorithm (CSA) which is specific type of AIAs has proven to be superior to other AIAs according to (Zandieh, Fatemi Ghomi, and Moattar Husseini 2006). CSA is thus used in this study. AIAs based on CSA possess two primary operators:

- Cloning selection.
- Affinity maturation.

For the former, those schedules that effectively minimise the F-JSSP are proliferated by cloning. Affinity maturation encompasses two basic phases: *hyper-mutation* and *receptor editing*. In the hyper-mutation phase, inferior schedules undergo higher rate of mutation as opposed to superior ones. Receptor editing manages the hyper-mutation procedure. The population evolves by a set of operators until some stopping criterion is met. For more detail on application of AIAs in production scheduling, see Zandieh, Fatemi Ghomi, and Moattar Husseini (2006). *Unlike the original AIAs*

*where only inferior schedules undergo hyper-mutation, with the proposed AISA, all schedules undergo hyper-mutation by applying a fast and effective SA algorithm.* The proposed hybrid AISA algorithm searches the problem space populated with encoded feasible schedules.

### 4.2 *Antibody (feasible schedule) representation*

F-JSSP is actually a combination of machine assignment and operations sequencing. A hierarchical approach with a novel encoding scheme consisting of two strings is utilised. With the sequencing string, all operations of a job are listed with the same symbol (number) and interpreted according to its already technically precedence-constrained operations set; hence, any permutation of these numbers is an operation sequence. Each job $j$ emerges $n_j$ times to represent its $n_j$ ordered operations. By scanning the permutation from left to right, the $k$th occurrence of a job number indicates the $k$-th operation in its processing route. The following example is provided to demonstrate how a feasible schedule consisting of sequencing string and assigning string is constructed for PF-JSSP. Table 3 shows the processing times of operations on each machine plus their respective feasible machines to execute them.

Considering the above table, an example of encoded sequencing is provided. Let us assume the initial random sequence is the following string $\{1,2,1,3,3,4,1,4,3,2\}$. The decoded sequence becomes:

$$\{O_{11}, O_{21}, O_{12}, O_{31}, O_{32}, O_{41}, O_{13}, O_{42}, O_{33}, O_{22}\}.$$

Consider the following three-row representation of the initial feasible sequence. Ten random numbers between [0,1] are equivalent to the number of operations to be generated, and each random number is assigned to one of the operations. After random numbers were generated, they are sorted in a non-decreasing order. Therefore, the sorted string is the initial feasible sequence. This operation is done using a SHIFT (single-point) operator (Naderi et al. 2009; Naderi, Zandieh, and Roshanaei 2009; Roshanaei, Seyyed Esfehani, and Zandieh 2010).

In order to generate new sequence out of the sorted initial solution (Table 4), one of the 10 random numbers is randomly selected and the value of which is randomly regenerated between [0,1]. As an example, for every corresponding previously generated random number (such as 0.51 in Table 5), another randomly number is to be regenerated for the newly generated sequence (0.19 in Table 6) and the whole random numbers are sorted in non-decreasing order again. As you can see, the first operation of job number 4 is relocated from cell 6 to cell 3. Below the newly constructed sequence is shown.

Thus far, the sequencing to take place has been explained. No machine has yet been assigned to each operation. Therefore, the second string shows the machine selected from the eligible machines to process each operation. Based on Table 3, a subset of machines with maximum number of machines per operation is defined. Therefore, for each row in Table 3, a number showing the maximum number of eligible machines is defined. Then, within the defined range, a random integer number representing the selected machine is generated. In the second string, based on the possible assignments of operations to machines, operation $O_{3,1}$ is processed by its first eligible machine, $M_1$, operation $O_{1,1}$ by machine 2 which is the first eligible machine for it, and operation $O_{2,1}$ is executed on machine 3 which is the second eligible machine for $O_{2,1}$ it, etc. Table 6, completely demonstrates how a feasible (antibody) schedule consisting of two strings is generated (Table 7).

Table 3. Processing times and possible assignments of operations to machines.

| Processing times | $M_1$ | $M_2$ | $M_3$ | Assignment | $M_1$ | $M_2$ | $M_3$ |
|---|---|---|---|---|---|---|---|
| $O_{11}$ | 30 | $\infty$ | 18 | $O_{11}$ | 1 | 0 | 1 |
| $O_{12}$ | $\infty$ | 40 | 40 | $O_{12}$ | 0 | 1 | 1 |
| $O_{13}$ | 34 | 40 | $\infty$ | $O_{13}$ | 1 | 1 | 0 |
| $O_{21}$ | $\infty$ | 60 | 65 | $O_{21}$ | 0 | 1 | 1 |
| $O_{22}$ | 66 | $\infty$ | 60 | $O_{22}$ | 1 | 0 | 1 |
| $O_{31}$ | $\infty$ | $\infty$ | 40 | $O_{31}$ | 0 | 0 | 1 |
| $O_{32}$ | 20 | 32 | $\infty$ | $O_{32}$ | 1 | 1 | 0 |
| $O_{33}$ | $\infty$ | 30 | 30 | $O_{33}$ | 0 | 1 | 1 |
| $O_{41}$ | $\infty$ | 62 | 40 | $O_{41}$ | 0 | 1 | 1 |
| $O_{42}$ | 50 | 60 | $\infty$ | $O_{42}$ | 1 | 1 | 0 |

Table 4. Generated initial solution.

| Rand no. | 0.06 | 0.95 | 0.12 | 0.89 | 0.76 | 0.32 | 0.99 | 0.23 | 0.47 | 0.51 |
|---|---|---|---|---|---|---|---|---|---|---|
| Operations | 1 | 3 | 2 | 4 | 1 | 3 | 2 | 1 | 3 | 4 |

Table 5. Sorted initial solution.

| Rand no. | 0.06 | 0.12 | 0.23 | 0.32 | 0.47 | *0.51* | 0.76 | 0.89 | 0.95 | 0.99 |
|---|---|---|---|---|---|---|---|---|---|---|
| Operations | 1 | 2 | 1 | 3 | 3 | 4 | 1 | 4 | 3 | 2 |

Table 6. Newly generated sequence based on initial solution.

| Rand no. | 0.06 | 0.12 | *0.19* | 0.23 | 0.32 | 0.47 | 0.76 | 0.89 | 0.95 | 0.99 |
|---|---|---|---|---|---|---|---|---|---|---|
| Operations | 1 $O_{11}$ | 2 $O_{21}$ | 4 $O_{41}$ | 1 $O_{12}$ | 3 $O_{31}$ | 3 $O_{32}$ | 1 $O_{13}$ | 4 $O_{42}$ | 3 $O_{33}$ | 2 $O_{22}$ |

Table 7. Representation of an antigen (a feasible schedule).

| Rand no. | 1 | 2 | 4 | 3 | 1 | 4 | 2 | 3 | 1 | 3 | Affinity = 1/170 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Operations | $O_{11}$ | $O_{21}$ | $O_{41}$ | $O_{31}$ | $O_{12}$ | $O_{42}$ | $O_{22}$ | $O_{32}$ | $O_{13}$ | $O_{33}$ | |
| Machines | 1 | 1 | 2 | 1 | 1 | 1 | 2 | 1 | 2 | 1 | Make-span = 170 |
| Process time | 30 | 60 | 40 | 40 | 40 | 50 | 60 | 20 | 40 | 30 | |

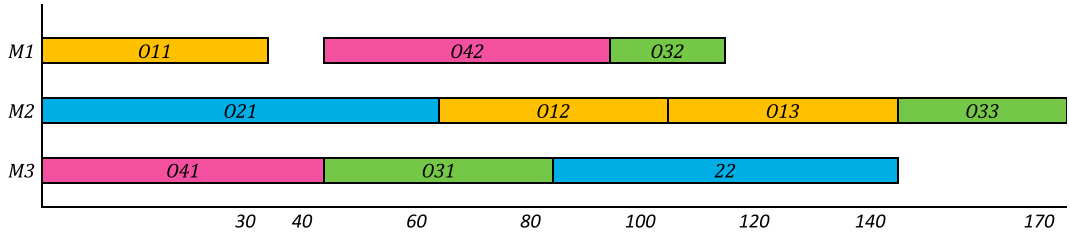


Figure 1. The unfolded antibody (feasible solution) in terms of a Gantt chart.

In Figure 1, the above feasible schedule is unfolded to its equivalent Gantt chart. The corresponding objective function for this feasible schedule is 170. Any regeneration of the random numbers for two strings generates a new feasible schedule with new objective function value. Each of these feasible schedules represents an antigen in the initial population of the AISA.

The initialisation is used for the random generation of schedules (popsize) from the feasible region. Once schedules have been made recognisable to the AISA, an *affinity* value is assigned to each schedule according to their performance – the more desirable the *schedule*, the higher is its affinity. Then, the schedules evolve by applying an effective SA to all existing antibodies until the stopping criterion is met. A typical iteration of AIA *generation* proceeds as follows: according to a newly customised *affinity function for the problem at hand*, the number of the schedule clones proliferated by each schedule generator is calculated and added to a *mutating pool*. A selection mechanism chooses the schedules in the current *mutating pool* where schedules with lower make-span values (higher affinity values) have more chance of being selected.

### 4.3 *Cloning selection procedure*

Schedules with lower make-span values have higher affinity values and are considered to be superior for solving PF-JSSP. But since the objective function is the minimisation of make-span, better schedules are those resulting in lower values of make-span. Therefore, the following function is defined to convert the objective function value of each schedule to its affinity value:

$$\text{Affinity } (t) = 1/\text{Make-span}(t) \tag{21}$$

Therefore, higher *affinity* values lead to lower objective function values. The probability of cloning each schedule to transfer into the *mutating pool* is directly proportional to its affinity value. The *mutating pool* has a fixed number of schedules (population size). The best schedule in a generation is copied to the next mutation pool. A hybrid *selection* mechanism is used (*popsize*-1) times and ranking and tournament selection Goldberg (1989) are applied each time a schedule is copied into the pool. The population of schedules evolve by a set of SA operators until some stopping criterion (*n.m.0.2* s) is met. The selected clones are hyper-mutated and generate new schedules (*off-springs*). The new population is evaluated, and the whole process is repeated. The complete procedure of AIA has been described in Figure 2.

### 4.4 *Affinity maturation procedure via SA*

All schedule clones in the pool undergo an operator called *hyper-mutation* which makes a random change in the schedule clones using a fast converging SA. The complete procedure of affinity maturation has been shown in Figure 3. Any meta-heuristic algorithm, in order to avoid getting trapped in local optima, should possess two built-in capabilities:

- *Exploration or diversification*: Alludes to the fact that a meta-heuristic is capable of visiting diverse promising regions in the search space by its operators. AIAs are very well known for this capability.
- *Exploitation or intensification*: Alludes to the capability of meta-heuristics to completely exploit prospective solutions in newly found regions using diversification operators. SAs are well established for doing this.

```
Procedure: Artificial immune algorithm (AIA)

Initialization phase (Random population (N) and parameters
Affinity evaluation phase
While stopping criterion is not met do
      Cloning selection phase
      Affinity maturation phase
      Affinity evaluation phase
Endwhile
```

Figure 2. Pseudo code of AIA.

```
Procedure: Affinity maturation phase via simulated annealing
(SA)
Initialization (the one clone in the mutating pool)
counter = 0
 while counter <= n do
     for i = m  do
Generate a new neighbor from current solution (shift operator)
Acceptance criterion
Update the best solution so far found
    endfor
    if the best solution is improved in this temperature do
counter = 0
     else
        counter = counter + 1
     endif
   Temperature reduction through exponential cooling scheme
Endwhile
If stopping criterion is met, return the best schedule
```

Figure 3. Pseudo code of SA.

Therefore, it has been seen that the combination of the flexible and effective population-based algorithm to search for the optimal solution and the convergent characteristics of SA provides the rationale for developing the proposed AISA strategy to schedule PF-JSSP while minimising make-span. After cloning schedules with lower make-span values, SA is applied to *all clones* in the mutated *pool*. For both sequencing and assigning phases, the SHIFT procedure that has proven to be superior to other schedule generators is applied (Naderi et al. 2009; Naderi, Zandieh, and Roshanaei 2009; Roshanaei, Seyyed Esfehani, and Zandieh 2010). First, based on the sequencing string, a new sequence of operations is constructed, and then, again the SHIFT procedure is used for operation assignment; ten random machine assignments are generated for each sequence and the best one is selected. The newly generated schedule ($s$) from the incumbent solution ($x$) is accepted if the following equation holds ($\Delta C = \text{make-span}(s) - \text{make-span}(x) \leq 0$; otherwise, schedule $s$ undergoes another probabilistic acceptance criterion which is $\exp -(\Delta C/t_i)$. Parameter $t$, called the temperature, controls the acceptance rule. At each temperature ($t_i$), the SA algorithm constructs 15 schedules under the exponential annealing scheme ($t_i = \alpha \cdot t_{i-1}$ where $\alpha \in (0, 1)$ is temperature decrement rate) and stops search explorations if no improvement is to be made after seven consecutive temperatures.

### 4.5 *AISA parameter tuning*

Many researchers since the introduction of meta-heuristics have accentuated the significance of parameter tuning on the performance improvement of algorithms. Naderi et al. (2009) and Naderi, Zandieh, and Roshanaei (2009) utilised a Taguchi's design of experiment technique for calibrating the parameters of SA. They investigated seven parameters of SA with different values. Zandieh, Fatemi Ghomi, and Moattar Husseini (2006) similarly investigated different parameters of AIAs. According to the literature survey for parameters tuning of the two algorithms, and also based on empirical tests, the following values for different parameters of AISA have been recognised (Table 8).

Among all combinations of parameters the following combination was selected: Both ranking selection and tournament mechanisms, random population of 70 antibodies, hierarchical two string encoding scheme, exponential cooling schedule, initial temperature of 20, number of iteration of 15, desired number of temperatures of 7, single-point (SHIFT) operator and certain computational time as stopping criterion were selected.

## 5. Comparative evaluations and discussions

This section addresses the numerical size complexity of each MILP model. Numerical and computational analyses are conducted based on the optimal, feasible and lower bound solutions of the enhanced MILPs. Finally, efficiency of each solution methodology in quantified. Section 5.1 has been allocated to numerically measure the size complexity of each MILP.

### 5.1 *Size complexity measurement and numerical evaluation of MILPs*

Addressing size complexity of mathematical models started with measuring size complexity of MILP models for flow-shop scheduling environment. This issue gained momentum by the end of the 1980s. Stafford (1988) and Stafford and Tseng (2005) measured size complexity of flow-shop scheduling problems. Liao and You (1992) proposed a new MILP for job shop scheduling problem that could dominate previously developed models from literature. They used size complexity measurement to prove the efficiency of their proposed MILP. Pan (1997) compiled all mathematical models

Table 8. Parameters of AISA.

| Parameters | Choice 1 | Choice 2 | Choice 3 | Selected choice |
|---|---|---|---|---|
| Selection mechanism | Tournament | Rank-based | Roulette wheel | Both 1 and 2 |
| Initial population | 30 | 50 | 70 | 70 |
| Encoding scheme | Triplet one-string | Hierarchical two-string | | Hierarchical |
| Cooling schedule | Linear | Hyperbolic | Exponential | Exponential |
| Initial temperature | 10 | 15 | 20 | 20 |
| Number of solutions in each ($t_i$) | 5 | 10 | 15 | 15 |
| Number of desired temperature between initial temperature and stopping temperature | 5 | 7 | 10 | 7 |
| Neighbourhood search structure | Shift (single-point) | Multi-point | Swap | Shift |
| Stopping criterion | Number of generations | CPU time | | CPU time |

developed under different modelling paradigms for permutation flow-shop, non-permutation flow-shop and job shop scheduling problems. Stafford and Tseng (2005) extended the results obtained by Pan (1997) to solve larger instances of the problem. Naderi and Ruiz (2010) proposed several mathematical models for distributed permutation flow shop scheduling problem. They also measured the complexity of each of their proposed MILPs parametrically.

For the purpose of numerically computing size complexity of different F-JSSP MILP models, they are rigorously analysed by applying them to the F-data set. Contributing factors to size complexity are:

- Binary integer decision variables (BIVs).
- Continuous decision variables (CVs).
- Numbers of constraints.

All the MILPs are applied to F-data set and their size complexity are measured based on 20 different instances. The traditional way of measuring the performance of mathematical models relied on counting their constituents that are:

- Binary integer decision variables (BIVs).
- Continuous decision variables (CVs).
- Numbers of constraints (ODs and DCs).

Recently, new measures for appraising the performances of MILPs have been introduced (Stafford and Tseng 2005). Sometimes, certain MILP possesses fewer numbers of BIVs, while higher numbers of constraints and other MILPs have higher number of BIVs but fewer numbers of constraints. The aforesaid performance measures could hardly serve as a credible basis to adjudicate whether the proposed MILPs surpass the best-performing MILP (Roshanaei, ElMaraghy, and Azab 2012). Hence, three other performance measures, namely size complexity, computational time and the quality of schedules generated by each MILP are exploited. These performance measures are as follows:

(1) The speed with which a certain problem is solved. This measure is referred to as computational or run time. The lower the time used, the better the MILP.
(2) The size complexity of the problem that is solved by the MILPs. The larger the size, the better the MILP.

Table 9. Numerical analyses of position-based MILPs.

| | | MILP-1 (Fattahi, Mehrabad, and Jolai 2007) | | | | | Proposed position-based MILP (MILP-4) | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Instance | Size | BIVs | CVs | NCs | CPU (s) | $C_{max}$ | BIVs | CVs | NCs | CPU (s) | $C_{max}$ |
| SFJS1 | 2.2.2 | 40 | 26 | 134 | **0** | **66**[a] | **32** | **17** | **114** | 0.01 | **66**[a] |
| SFJS2 | 2.2.2 | 32 | 24 | 108 | **0** | **107**[a] | **24** | **15** | **92** | 0.02 | **107**[a] |
| SFJS3 | 3.2.2 | 72 | 36 | 234 | 7 | **221**[a] | **60** | **22** | **188** | **0.22** | **221**[a] |
| SFJS4 | 3.2.2 | 84 | 38 | 236 | 11 | **355**[a] | **60** | **22** | **188** | **1.20** | **355**[a] |
| SFJS5 | 3.2.2 | 84 | 38 | 272 | 87 | **119**[a] | **72** | **24** | **218** | **0.98** | **119**[a] |
| SFJS6 | 3.3.3 | 189 | 50 | 497 | 129 | **320**[a] | **135** | **31** | **378** | **0.22** | **320**[a] |
| SFJS7 | 3.3.5 | 225 | 55 | 598 | 135 | **397**[a] | **162** | **36** | **461** | **0.05** | **397**[a] |
| SFJS8 | 3.3.4 | 216 | 55 | 589 | 116 | **253**[a] | **162** | **28** | **437** | **4.85** | **253**[a] |
| SFJS9 | 3.3.3 | 243 | 56 | 584 | 319 | **210**[a] | **162** | **34** | **441** | **0.14** | **210**[a] |
| SFJS10 | 4.3.5 | 300 | 66 | 862 | 510 | **516**[a] | **240** | **42** | **649** | **3.92** | **516**[a] |
| MFJS1 | 5.3.6 | 720 | 99 | 1829 | 3600 | 470[b] | **495** | **60** | **1240** | **600** | **468**[b] |
| MFJS2 | 5.3.7 | 840 | 106 | 1986 | 3600 | 484[b] | **585** | **67** | **1454** | **600** | **446**[b] |
| MFJS3 | 6.3.7 | 1260 | 131 | 2819 | 3600 | 564[b] | **846** | **66** | **2006** | **600** | **466**[b] |
| MFJS4 | 7.3.7 | 1617 | 149 | 3789 | 3600 | 684[b] | **1176** | **92** | **2751** | **600** | **565**[b] |
| MFJS5 | 7.3.7 | 1617 | 149 | 3726 | 3600 | 696[b] | **1113** | **89** | **2616** | **600** | **514**[b] |
| MFJS6 | 8.3.7 | 2184 | 174 | 4766 | 3600 | 786[b] | **1512** | **103** | **3476** | **600** | **616**[b] |
| MFJS7 | 8.4.7 | 3584 | 219 | 7883 | 3600 | 1433[b] | **2496** | **111** | **5539** | **3600** | **881**[b] |
| MFJS8 | 9.4.8 | 4896 | 256 | 9778 | 3600 | 1914[b] | **3096** | **140** | **6872** | **4617** | **916**[b] |
| MFJS9 | 11.4.8 | 7040 | 308 | 14,190 | 3600 | 764[LB] | **4532** | **167** | **9887** | **3600** | 764[LB] |
| MFJS10 | 12.4.8 | 8832 | 346 | 16,784 | 3600 | 944[LB] | **5373** | **181** | **11,648** | **3600** | 944[LB] |
| Total | | 34,075 | 2381 | 71,664 | 37,314 | | **22,333** | **1347** | **50,655** | 19,028.91 | |

Note: LB: lower bound. Boldfacing shows the best element in each row or best solution.
[a]Optimal solution.
[b]Best feasible integer.

(3) The quality of schedules that are generated by the MILPs. This measure is computed through calculating objective function values. Since the objective functions of scheduling problems are principally of minimisation nature, the lower this value, the better the MILP.

In this study, all of the above-mentioned performance measures are examined to determine the best thus far proposed position- and sequence-based MILPs. MILPs are coded in CPLEX 12.1 optimisation software package. Experiments are carried out on a laptop with the following specifications: Core (TM)2 with Quad CPU of 2.83 GHz processor and 4 GB Ram. In this study, comparative evaluations among F-JSSP MILPs start with contrasting the performances of position-based MILPs; Tables 9 and 10 have been designed for the purposes. Table 9 reports the structural elements of position-based MILPs (MILP-1 and MILP-4) and Table 10 reports their generated relative percentage deviation (RPD) on all 20 instances of F-data set.

MILP-1 produced total number of 34,075 BIVs vis-à-vis the MILP-4 with total numbers of 22,333 BIVs on 20 instances of F-data set. While MILP-1 generated 2381 numbers of CVs, the MILP-4 produced 1347CVs. With regards to the number of constraints, the MILP-1 generated 71,664 numbers of constraints, while the MILP-4 did 50,655. MILP-1 used computational time of 37,314 to solve 20 instances of F-data, while MILP-4 used 19,028.91. The MILP-1 produced 96.1% of RPD in comparison with the MILP-4 which did 0% of RPD on computational time. The RPD associated with computational time is computed based on the total amount of time it used and it is not therefore calculated on each instance of F-data set. Due to the structural efficiency of the MILP-4, it produced eight new enhanced feasible integer solutions on MFJS-1 up to MFJS-8. Table 9 reports RPDs of each MILP on each instance of the F-data set. Other than the RPD associated with computational time, all other performance measures are investigated in Table 10. Detailed evaluations of position-based MILPs are given in Table 10.

In Table 10, position-based MILPs (MILP-1 and MILP-4) are rigorously analysed based on their produced BIVs, CVs, NCs, and also quality of generated schedules that are judged by $C_{\max}$ values produced for each instance of the problem. The most determining factor affecting performance of MILPs is the number of BIVs they generate (Pan 1997). If two MILPs have the same number of BIVs, (Liao and You 1992) and (Wilson 1989) showed that the next influential element is the number of constraints. However, Roshanaei, ElMaraghy, and Azab (2012) showed that even having fewer numbers of continuous variables play active role in enhancing the performances of MILPs. The MILP-1 and MILP-4 produced 34,075 and 22,333 numbers of BIVs, respectively. The MILP-4 yielded fewer BIVs on all 20 instances of the problem. The number of BIVs that the MILP-4 generates on each instance of the problem is less than that of MILP-1.

Table 10. Numerical analyses of position-based MILP models (MILP-1 and MILP-4).

| | BIVs-1 | BIVs-4 | RPD-1% | CVs-1 | CVs-4 | RPD-4% | NCs-1 | NCs-4 | RPD-4% | $C_{\max-1}$ | $C_{\max-4}$ | RPD-4% |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SFJS1 | 40 | **32** | 20 | 26 | **17** | **52.94** | 134 | **114** | **17.54** | **66**[a] | **66**[a] | **0** |
| SFJS2 | 32 | **24** | 25 | 24 | **15** | **60** | 108 | **92** | **17.39** | **107**[a] | **107**[a] | **0** |
| SFJS3 | 72 | **60** | 20 | 36 | **22** | **63.63** | 234 | **188** | **24.46** | **221**[a] | **221**[a] | **0** |
| SFJS4 | 84 | **60** | 40 | 38 | **22** | **72.72** | 236 | **188** | **25.53** | **355**[a] | **355**[a] | **0** |
| SFJS5 | 84 | **72** | 20 | 38 | **24** | **58.33** | 272 | **218** | **24.77** | **119**[a] | **119**[a] | **0** |
| SFJS6 | 189 | **135** | 35.29 | 50 | **31** | **61.29** | 497 | **378** | **31.48** | **320**[a] | **320**[a] | **0** |
| SFJS7 | 225 | **162** | 38.88 | 55 | **36** | **52.77** | 598 | **461** | **29.71** | **397**[a] | **397**[a] | **0** |
| SFJS8 | 216 | **162** | 33.3 | 55 | **28** | **96.42** | 589 | **437** | **34.78** | **253**[a] | **253**[a] | **0** |
| SFJS9 | 243 | **162** | 50 | 56 | **34** | **64.7** | 584 | **441** | **32.42** | **210**[a] | **210**[a] | **0** |
| SFJS10 | 300 | **240** | 25 | 66 | **42** | **57.14** | 862 | **649** | **32.82** | **516**[a] | **516**[a] | **0** |
| MFJS1 | 720 | **495** | 45.45 | 99 | **60** | **65** | 1829 | **1240** | 47.5 | 470[b] | **468**[b] | 0.42 |
| MFJS2 | 840 | **585** | 43.59 | 106 | **67** | **58.2** | 1986 | **1454** | 36.59 | 484[b] | **446**[b] | 8.52 |
| MFJS3 | 1260 | **846** | 48.93 | 131 | **66** | **98.48** | 2819 | **2006** | 40.53 | 564[b] | **466**[b] | 21.03 |
| MFJS4 | 1617 | **1176** | 37.5 | 149 | **92** | **61.95** | 3789 | **2751** | 37.73 | 684[b] | **565**[b] | 21.06 |
| MFJS5 | 1617 | **1113** | 45.28 | 149 | **89** | **67.41** | 3726 | **2616** | 42.43 | 696[b] | **514**[b] | 35.4 |
| MFJS6 | 2184 | **1512** | 44.4 | 174 | **103** | **68.93** | 4766 | **3476** | 37.11 | 786[b] | **616**[b] | 27.59 |
| MFJS7 | 3584 | **2496** | 43.59 | 219 | **111** | **97.3** | 7883 | **5539** | 42.32 | 1433[b] | **881**[b] | 62.65 |
| MFJS8 | 4896 | **3096** | 58.13 | 256 | **140** | **85.85** | 9778 | **6872** | 42.29 | 1914[b] | **916**[b] | 108.95 |
| MFJS9 | 7040 | **4532** | 55.33 | 308 | **167** | **84.43** | 14,190 | **9887** | 43.52 | 764[LB] | **764**[LB] | **0** |
| MFJS10 | 8832 | **5373** | 64.37 | 346 | **181** | **91.16** | 16,784 | **11,648** | 44.1 | 944[LB] | **944**[LB] | **0** |
| Total | 34,075 | **22,333** | | 2381 | **1347** | | 71,664 | 50,655 | | 17,463 | 12,618 | |
| Average RPD of MILP-F | | | 39.70% | | | 70.93% | | | 34.25% | | | 14.28% |

Note: Boldfacing shows the best element in each row or best solution.
[a]Optimal solution.
[b]Best feasible integer.

The MILP-4 handles sequencing and assignment sub-problems in F-JSSP with only one type of BIV, while MILP-1 utilises two types of BIVs; one for assignment of operations to machines and second for sequencing of operations on different machines. As a result, the MILP-1 produced 39.70% of RPD. From computational perspective, this improvement by itself drastically enhances the efficiency and effectiveness of MILPs. But the MILP-4 has more to offer. It has much fewer CVs and NCs as opposed to MILP-1. The MILP-4 solves all the 20 instances of F-data set with fewer numbers of CVs. The MILP-1 solved the benchmark with 2381 CVs vis-à-vis MILP-4 with the 1347 CVs. This improvement in the proposed MILP is attributed to the utilisation of fewer types of continuous variables. The MILP-1 utilised four types of CVs, while the MILP-4 used only two types of CVs. Consequently, the MILP-1 produces 70.93% of RPD on CVs. As Table 10 demonstrates, MILP-1 requires more constraints to solve the benchmark. MILP-4 and MILP-1 used 50,655 and 71,644 NCs to solve 20 instances of the benchmark, respectively. It causes the MILP-1 to generate 34.25% of RPD. So far the performances of position-based MILPs have been investigated based on their structural complexity constituents (BIVs, CVs and NCs). Obviously, the MILP with fewer numbers of structural constituents produces schedules with better quality or at least with the same quality. The ultimate criterion for comparing the effectiveness of MILPs is the quality of generated schedules. Thus, particular attention is paid to this performance measure. The MILP-1 obtained 10 optimal solutions in the first instances of the F-data set (SFJS1–SFJS10), so does MILP-4. It appears that size complexity does not interfere with the quality of solutions up to SFJS10. Both position-based MILPs are capable of exhaustively visiting existing nodes and coming up with optimal solutions to the first 10 instances of the problem. Size complexity exerts its prohibitive role right after SFJS10. Starting from MFJS-1 to MFJS8, MILP-1 produces feasible integer solutions that are far inferior to those of the MILP-4. These performance differences could be largely anticipated based on the structures of MILPs. Now it is safe to state that the solution quality of MILPs is inversely proportional to the numbers of BIVs, CVs and NCs. Given the structural improvements that MILP-4 has made over MILP-1, having a 14.28% RPD for MILP-1 for the quality of generated schedules is quite natural. Table 11 shows numerical comparison between the MILP-2 and MILP-5.

As can be seen in Table 12, the MILP-2 and the proposed sequence-based MILP (MILP-5) have been contrasted on all performance measures. The MILP-5 and MILP-2 produce 3317 and 4202 of BIVs, respectively. The MILP-2 and MILP-5 generate 1741 and 824 numbers of CVs, respectively. The MILP-2 and MILP-5 generate 9207 and 9090 numbers of constraints. As can be seen, the MILP-5 outperforms the MILP-2 in all structural elements. The MILP-2 and

Table 11. Numerical comparison between the MILP-2 and the MILP-5.

| | | MILP-2 (Ozguven, Ozbakir, and Yavuz 2010) | | | | | MILP-5 (Proposed sequence-based MILP) | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Instance | Size | BIVs | CVs | NCs | CPU (s) | $C_{max}$ | BIVs | CVs | NCs | CPU (s) | $C_{max}$ |
| SFJS1 | 2.2.2 | 16 | 19 | 42 | 0.02 | **66**[a] | **12** | **9** | **40** | 0.02 | **66**[a] |
| SFJS2 | 2.2.2 | 10 | 15 | 30 | 0.00 | **107**[a] | **9** | **7** | **28** | 0.02 | **107**[a] |
| SFJS3 | 3.2.2 | 26 | 24 | 67 | 0.02 | **221**[a] | **21** | **11** | **64** | 0.01 | **221**[a] |
| SFJS4 | 3.2.2 | 26 | 24 | 67 | 0.00 | **355**[a] | **22** | **11** | **64** | 0.01 | **355**[a] |
| SFJS5 | 3.2.2 | 36 | 28 | 87 | 0.06 | **119**[a] | **24** | **13** | **84** | 0.05 | **119**[a] |
| SFJS6 | 3.3.3 | 39 | 34 | 99 | **0.03** | **320**[a] | **34** | **16** | **96** | 0.03 | **320**[a] |
| SFJS7 | 3.3.5 | 36 | 40 | 93 | **0.02** | **397**[a] | **35** | **19** | **94** | 0.02 | **397**[a] |
| SFJS8 | 3.3.4 | 45 | 40 | 111 | **0.02** | **253**[a] | **40** | **19** | **108** | 0.06 | **253**[a] |
| SFJS9 | 3.3.3 | 55 | 40 | 131 | **0.03** | **210**[a] | **45** | **19** | **128** | 0.03 | **210**[a] |
| SFJS10 | 4.3.5 | 48 | 45 | 124 | **0.02** | **516**[a] | **46** | **21** | **120** | 0.03 | **516**[a] |
| MFJS1 | 5.3.6 | 103 | 72 | 241 | 0.44 | **468**[a] | **84** | **34** | **236** | 0.73 | **468**[a] |
| MFJS2 | 5.3.7 | 128 | 84 | **291** | 6.49 | **446**[a] | **101** | **40** | 294 | 1.46 | **446**[a] |
| MFJS3 | 6.3.7 | 190 | 103 | 422 | 4.14 | **466**[a] | **141** | **48** | **408** | 1.03 | **466**[a] |
| MFJS4 | 7.3.7 | 250 | 120 | 549 | 1779 | 564[b] | **184** | **57** | **542** | 245 | **554**[a] |
| MFJS5 | 7.3.7 | 243 | 118 | 535 | 50.98 | **514**[a] | **184** | **54** | **502** | 13.12 | **514**[a] |
| MFJS6 | 8.3.7 | 307 | 133 | **670** | 3600 | 635[b] | **240** | **64** | **682** | 8403 | **608**[a] |
| MFJS7 | 8.4.7 | 475 | 165 | 1022 | 3600 | 935[b] | **364** | **79** | **1014** | **60** | **881**[b] |
| MFJS8 | 9.4.8 | 519 | 182 | 1119 | 3600 | 905[b] | **408** | **86** | **1088** | **180** | **894**[b] |
| MFJS9 | 11.4.8 | 751 | 218 | 1601 | 3600 | 1192[b] | **604** | **104** | **1598** | **300** | **1192**[b] |
| MFJS10 | 12.4.8 | 899 | 237 | 1906 | 3600 | 1276[b] | **719** | **113** | **1900** | **3600** | **1276**[b] |
| Total | | 4202 | 1741 | 9207 | 19,841.3 | | **3317** | **824** | **9090** | **12,804.62** | |

Note: Boldfacing shows the best element in each row or best solution.
[a]Optimal solution.
[b]Best obtained so-far feasible solution.

Table 12. Numerical analyses between the MILP-2 and the MILP-5.

| Instance | BIVs-2 | BIVs-5 | RPD2% | CVs-2 | CVs-5 | RPD-2% | NCs-2 | NCs-5 | RPD-2% | RPD-5% | $C_{max}$-2 | *Cmax*-5 | RPD-2% |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SFJS1 | 16 | **12** | 33.3 | 19 | **9** | **111** | 42 | **40** | 5 | **0** | **66**[a] | **66**[a] | **0** |
| SFJS2 | 10 | **9** | 11.1 | 15 | **7** | **114** | 30 | **28** | 7.14 | **0** | **107**[a] | **107**[a] | **0** |
| SFJS3 | 26 | **21** | 23.8 | 24 | **11** | **118** | 67 | **64** | 4.68 | **0** | **221**[a] | **221**[a] | **0** |
| SFJS4 | 26 | **22** | 18.18 | 24 | **11** | **118** | 67 | **64** | 4.68 | **0** | **355**[a] | **355**[a] | **0** |
| SFJS5 | 36 | **24** | 50 | 28 | **13** | **115** | 87 | **84** | 3.57 | **0** | **119**[a] | **119**[a] | **0** |
| SFJS6 | 39 | **34** | 14.7 | 34 | **16** | **113** | 99 | **96** | 3.13 | **0** | **320**[a] | **320**[a] | **0** |
| SFJS7 | 36 | **35** | 2.8 | 40 | **19** | **111** | 93 | **94** | **0** | **1.07** | **397**[a] | **397**[a] | **0** |
| SFJS8 | 45 | **40** | 12.5 | 40 | **19** | **111** | 111 | **108** | 2.7 | **0** | **253**[a] | **253**[a] | **0** |
| SFJS9 | 55 | **45** | 22.2 | 40 | **19** | **111** | 131 | **128** | 2.34 | **0** | **210**[a] | **210**[a] | **0** |
| SFJS10 | 48 | **46** | 4.35 | 45 | **21** | **114** | 124 | **120** | 3.33 | **0** | **516**[a] | **516**[a] | **0** |
| MFJS1 | 103 | **84** | 22.6 | 72 | **34** | **112** | 241 | **236** | 2.12 | **0** | **468**[a] | **468**[a] | **0** |
| MFJS2 | 128 | **101** | 26.73 | 84 | **40** | **110** | 291 | 294 | **0** | 1.03 | **446**[a] | **446**[a] | **0** |
| MFJS3 | 190 | **141** | 34.75 | 103 | **48** | **115** | 422 | **408** | **3.43** | **0** | **466**[a] | **466**[a] | **0** |
| MFJS4 | 250 | **184** | 35.87 | 120 | **57** | **111** | 549 | **542** | **1.29** | **0** | 564[b] | **554**[a] | **1.8** |
| MFJS5 | 243 | **184** | 32.65 | 118 | **54** | **119** | 535 | **502** | **6.57** | **0** | **514**[a] | **514**[a] | **0** |
| MFJS6 | 307 | **240** | 27.91 | 133 | **64** | **108** | 670 | 682 | **1.79** | **0** | 635[b] | **608**[a] | **4.44** |
| MFJS7 | 475 | **364** | 30.19 | 165 | **79** | **109** | 1022 | **1014** | **0.79** | **0** | 935[b] | **881**[b] | **6.13** |
| MFJS8 | 519 | **408** | 27.2 | 182 | **86** | **112** | 1119 | **1088** | **2.85** | **0** | 905[b] | **894**[b] | **1.23** |
| MFJS9 | 751 | **604** | 24.34 | 218 | **104** | **110** | 1601 | **1598** | **0.19** | **0** | **1192**[b] | **1192**[b] | **0** |
| MFJS10 | 899 | **719** | 25.03 | 237 | **113** | **124** | 1906 | **1900** | **0.31** | **0** | **1276**[b] | **1276**[b] | **0** |
| Total | 4202 | **3317** | | 1741 | **824** | | 9207 | **9090** | | | | | |
| Average RPD of MILP-F | | | 24.01 | | | 113.3 | | | 2.8 | 0.11 | | | 0.68 |

Note: Boldfacing shows the best element in each row or best solution.
[a]Optimal solution.
[b]Best feasible integer.

MILP-5 consumed 19,841.3 and 12,804.62 s on 20 instances of F-data, respectively. Produced RDP of both MILPs is addressed in Table 12.

As Table 12 shows, the MILP-5 produces fewer numbers of BIVs on all 20 instances of F-data vs. the MILP-2. The MILP-2 generates RPD of 24.01% on number of BIVs. This shows the strengths of the MILP-5 vis-a-vis the MILP-2. In terms of number of CVs, the MILP-5 and the MILP-2 generate 824 and 1741 number of CVs. This number shows that the MILP-2 produces RPD of 113.3%. The MILP-5 produces fewer numbers of CVs on all 20 instances of F-data. With regards to number of constraints, the MILP-5 also outperforms the MILP-2. But this difference is not substantial. Both MILP-2 and MILP-5 generate RPDs of 2.8 and 0.11%, respectively. Having evaluated the structural efficiencies of the MILP-2 and MILP-5, their impacts on the solution effectiveness and computational efficiency of both MILPs are explored. Both MILPs obtain optimal solutions on 14 instances of F-data consisting of the first 13 instances of the problem (up to MFJS3) plus MFJS5. The MILP-5 obtains four new enhanced solutions as opposed to the MILP-2 in the following instances: MFJS4, MFJS6, MFJS7 and MFJS8. Solutions obtained for MFJS4 and MFJS6 are newly obtained optimal solutions, while solutions obtained for MFJS7 and MFJS8 are the enhanced feasible integer solutions. Both MILPs obtained the same feasible integer solutions for MFJS9 and MFJS10 within allotted computational time. Therefore, the MILP-2 produces RPD of 0.68% on quality of solutions.

Table 13 compares the building blocks of the MILP-5 with state-of-the-art MILP (MILP-3) proposed by (Roshanaei, ElMaraghy, and Azab 2012). Solutionwise, MILP-3 proved to be stronger than the MILP-2. Roshanaei, ElMaraghy, and Azab (2012) showed that MILP-3 possessed fewer numbers of CVs as opposed to the MILP-2. MILP-2 had fewer numbers of NCs and BIVs in turn. In Table 13, proposed MILP-5 is compared with MILP-3 for detailed performance comparison.

MILP-3 and MILP-5 produces 6389 and 3317 of numbers of BIVs, respectively. In terms of BIVs, MILP-5 is much stronger than MILP-3. With regards to number of CVs, MILP-3 produces fewer numbers of CVs. MILP-3 generates 472 numbers of CVs as compared to MILP-5 with 824 CVs. In this respect, MILP-3 is stronger than MILP-5. Concerning NCs, MILP-5 substantially outperforms MILP-3. MILP-5 generates 64,540, while MILP-3 does 9090. The impact of structural elements between MILP-3 and MILP-5 is explored on their solution effectiveness and computational efficiency.

Table 14 reports RDP for MILP-3 and MILP-5. MILP-3 produces RPD of 68.95% for number of BIVs. This simply means that MILP-5 produces fewer numbers of BIVs. MILP-5 produces RPD of 53.67% in terms of number of CVs.

Table 13. Numerical analyses between the MILP-3 and the MILP-5.

| | | MILP-3 (Roshanaei, ElMaraghy, and Azab 2012) | | | | | MILP-5 (Proposed sequence-based MILP) | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Instance | Size | BIVs | CVs | NCs | CPU (s) | $C_{max}$ | BIVs | CVs | NCs | CPU (s) | $C_{max}$ |
| SFJS1 | 2.2.2 | **12** | **7** | **40** | **0.02** | **66**[a] | **12** | 9 | **40** | 0.02 | **66**[a] |
| SFJS2 | 2.2.2 | 12 | **7** | 40 | **0.00** | **107**[a] | **9** | 7 | **28** | 0.02 | **107**[a] |
| SFJS3 | 3.2.2 | 24 | **10** | 84 | **0.02** | **221**[a] | **21** | 11 | **64** | 0.01 | **221**[a] |
| SFJS4 | 3.2.2 | 24 | **10** | 84 | 0.01 | **355**[a] | **22** | 11 | **64** | 0.01 | **355**[a] |
| SFJS5 | 3.2.2 | **24** | **10** | **84** | **0.03** | **119**[a] | **24** | 13 | **84** | 0.05 | **119**[a] |
| SFJS6 | 3.3.3 | 54 | **13** | 222 | **0.03** | **320**[a] | **34** | 16 | **96** | 0.03 | **320**[a] |
| SFJS7 | 3.3.5 | 72 | **13** | 348 | **0.02** | **397**[a] | **35** | 19 | **94** | 0.02 | **397**[a] |
| SFJS8 | 3.3.4 | 63 | **13** | 285 | **0.02** | **253**[a] | **40** | 19 | **108** | 0.06 | **253**[a] |
| SFJS9 | 3.3.3 | 54 | **13** | 222 | **0.03** | **210**[a] | **45** | 19 | **128** | 0.03 | **210**[a] |
| SFJS10 | 4.3.5 | 114 | **17** | 644 | 0.06 | **516**[a] | **46** | 21 | **120** | 0.03 | **516**[a] |
| MFJS1 | 5.3.6 | 180 | **21** | 1225 | 0.47 | **468**[a] | **84** | 34 | **236** | 0.73 | **468**[a] |
| MFJS2 | 5.3.7 | 195 | **21** | 1420 | **1.33** | **446**[a] | **101** | 40 | 294 | 1.46 | **446**[a] |
| MFJS3 | 6.3.7 | 261 | **25** | 2082 | 4.49 | **466**[a] | **141** | 48 | **408** | 1.03 | **466**[a] |
| MFJS4 | 7.3.7 | 336 | **26** | 2870 | **960** | **554**[a] | **184** | 57 | **542** | 245 | **554**[a] |
| MFJS5 | 7.3.7 | 336 | **29** | 2870 | **6.80** | **514**[a] | **184** | 54 | **502** | 13.12 | **514**[a] |
| MFJS6 | 8.3.7 | 420 | **33** | 3748 | **680.6** | **608**[a] | **240** | 64 | 682 | 8403 | **608**[a] |
| MFJS7 | 8.4.7 | 672 | **41** | 6608 | **1800** | **881**[b] | **364** | 79 | **1014** | **60** | **881**[b] |
| MFJS8 | 9.4.8 | 864 | **46** | 9630 | **1800** | **895**[b] | **408** | 86 | **1088** | **180** | **894**[b] |
| MFJS9 | 11.4.8 | 1232 | **56** | 14,586 | 1800 | **1192**[b] | **604** | 104 | **1598** | **300** | **1192**[b] |
| MFJS10 | 12.4.8 | 1440 | **61** | 17,448 | 3600 | **1276**[b] | **719** | 113 | **1900** | **3600** | **1276**[b] |
| Total | | 6389 | **472** | 64,540 | 10653.93 | | **3317** | 824 | 9090 | 12,804.62 | |

Note: Boldfacing shows the best element in each row or best solution.
[a]Optimal solution.
[b]Best feasible integer.

Table 14. Numerical analyses between the MILP-3 and MILP-5.

| Instance | BIVs2 | BIVs5 | RPD2% | CVs-2 | CVs-5 | RPD-5% | NCs-2 | NCs-5 | RPD-2% | $C_{max}$-2 | $C_{max}$-5 | RPD-2% |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SFJS1 | **12** | **12** | 0 | **7** | 9 | **22.2** | **40** | **40** | 0 | **66**[a] | **66**[a] | **0** |
| SFJS2 | 12 | **9** | 33.3 | **7** | 7 | **0** | 40 | **28** | 42.86 | **107**[a] | **107**[a] | **0** |
| SFJS3 | 24 | **21** | 14.28 | **10** | 11 | **10** | 84 | **64** | 31.25 | **221**[a] | **221**[a] | **0** |
| SFJS4 | 24 | **22** | 8.33 | **10** | 11 | **10** | 84 | **64** | 31.25 | **355**[a] | **355**[a] | **0** |
| SFJS5 | **24** | **24** | 0 | **10** | 13 | **30** | **84** | **84** | 31.25 | **119**[a] | **119**[a] | **0** |
| SFJS6 | 54 | **34** | 58.8 | **13** | 16 | **23.07** | 222 | **96** | 131.25 | **320**[a] | **320**[a] | **0** |
| SFJS7 | 72 | **35** | 105.7 | **13** | 19 | **46.15** | 348 | **94** | 270.2 | **397**[a] | **397**[a] | **0** |
| SFJS8 | 63 | **40** | 57.5 | **13** | 19 | **46.15** | 285 | **108** | 163.9 | **253**[a] | **253**[a] | **0** |
| SFJS9 | 54 | **45** | 20 | **13** | 19 | **46.15** | 222 | **128** | 73.44 | **210**[a] | **210**[a] | **0** |
| SFJS10 | 114 | **46** | 147.8 | **17** | 21 | **23.53** | 644 | **120** | 436.6 | **516**[a] | **516**[a] | **0** |
| MFJS1 | 180 | **84** | 114.3 | **21** | 34 | **61.9** | 1225 | **236** | 419 | **468**[a] | **468**[a] | **0** |
| MFJS2 | 195 | **101** | 93.06 | **21** | 40 | **47.5** | 1420 | 294 | 383 | **446**[a] | **446**[a] | **0** |
| MFJS3 | 261 | **141** | 85.1 | **25** | 48 | **92** | 2082 | **408** | **410.3** | **466**[a] | **466**[a] | **0** |
| MFJS4 | 336 | **184** | 82.6 | **26** | 57 | **119.2** | 2870 | **542** | **429.5** | **554**[a] | **554**[a] | **0** |
| MFJS5 | 336 | **184** | 82.6 | **29** | 54 | **86.2** | 2870 | **502** | **471.7** | **514**[a] | **514**[a] | **0** |
| MFJS6 | 420 | **240** | 75 | **33** | 64 | **93.93** | 3748 | 682 | **449.6** | **608**[a] | **608**[a] | **0** |
| MFJS7 | 672 | **364** | 84.62 | **41** | 79 | **92.68** | 6608 | **1014** | **551.7** | **881**[b] | **881**[b] | **0** |
| MFJS8 | 864 | **408** | 111.8 | **46** | 86 | **46.51** | 9630 | **1088** | **785.1** | **895**[b] | **894**[b] | **0.11** |
| MFJS9 | 1232 | **604** | 104 | **56** | 104 | **91.07** | 14,586 | **1598** | **812.8** | **1192**[b] | **1192**[b] | **0** |
| MFJS10 | 1440 | **719** | 100.3 | **61** | 113 | **85.25** | 17,448 | **1900** | **818.3** | **1276**[b] | **1276**[b] | **0** |
| Total | 6389 | **3317** | | **472** | 824 | | 64,540 | **9090** | | | | |
| Average RPDs | | | 68.95 | | | 53.67 | | | 337.15 | | | 0.0055 |

Note: Boldfacing shows the best element in each row or best solution.
[a]Optimal solution.
[b]Best feasible integer.

MILP-3 produces RPD of 337.15% on number of constraints. All in all, the proposed MILP (MILP-5) outperform MILP-3 in terms of number of BIVs and NCs.

By looking at Table 14, it is realised that MILP-5 obtains a better solution than MILP-3 on MFJS8. MILP-3 obtains the objective function value of 895, while MILP-5 obtains 894. For all other 19 instances of F-data, both MILPs obtain same result.

MILPs were measured for all the above-mentioned performance measures. Below, the strengths of MILPs are addressed for all performance measures. First the performances of MILPs are explored for the traditional performance measures:

- Number of BIVs: the MILP-5, MILP-2, MILP-3, MILP-4 and MILP-1 are ranked 1st–5th in terms of numbers of BIVs (ascending order). MILP-5, MILP-2, MILP-3, MILP-4 and MILP-1 generated 3317, 4202, 6389, 22,333 and 34,075, respectively. Figure 4 demonstrates the number of BIVs that each MILP generates.
- Number of CVs: MILP-3, MILP-5, MILP-4, MILP-2 and MILP-1 are ranked 1st–5th in terms of numbers of CVs (ascending order). MILP-3, MILP-5, MILP-4, MILP-2 and MILP-1 generated 472, 824, 1347, 1741 and 2381 number of CVs, respectively. Figure 5 demonstrates the number of CVs that each MILP generates.
- Number of Constraints: MILP-5, MILP-2, MILP-4, MILP-3 and MILP-1 are ranked 1$^{st}$–5th in terms of numbers of constraints (ascending order). The MILP-5, MILP-2, MILP-4, MILP-3 and MILP-1 generated 9090, 9207, 50,655, 64,540 and 71,664 numbers of constraints, respectively. Figure 6 demonstrates the number of constraints that each MILP generates.

Thus far, by comparing MILPs with traditional performance measures, the proposed position-based MILP-4 outperformed its equivalent position-based MILP-1 from literature. Likewise, the proposed sequence-based MILP-5 outperformed both MILP-2 and MILP-3 from literature.

The combined effect of such improvement should reflect itself in the modern performance measures. The analyses regarding new performance measures are provided next.

- *Computational time*: MILP-3, MILP-5, MILP-4, MILP-2 and MILP-1 are ranked 1st–5th in terms of computational efficiency; they used 10653.53, 12804.62, 19841.3, 19028.91 and 37,314 s of CUP time, respectively.

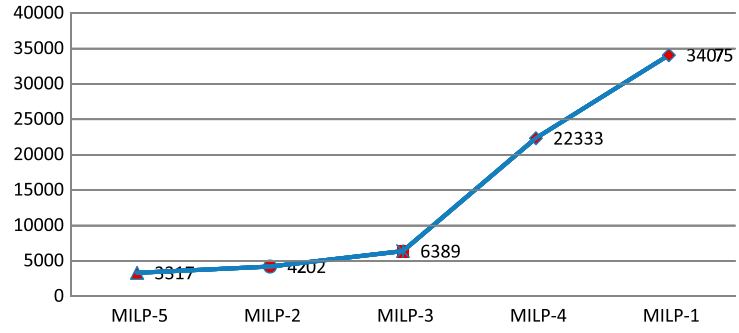


Figure 4. Performance evaluation of MILPs based on their BIVs.

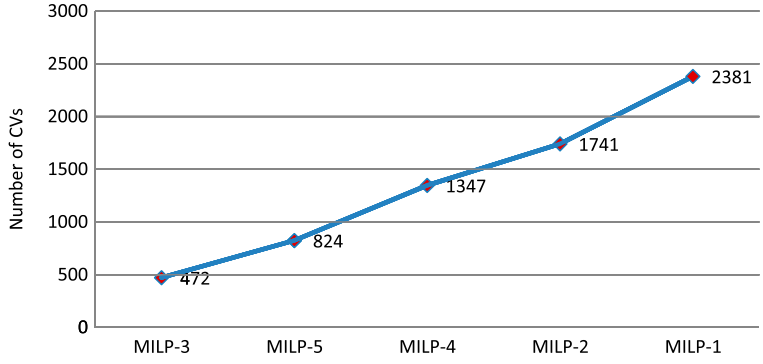


Figure 5. Performance evaluation of MILPs based on their CVs.

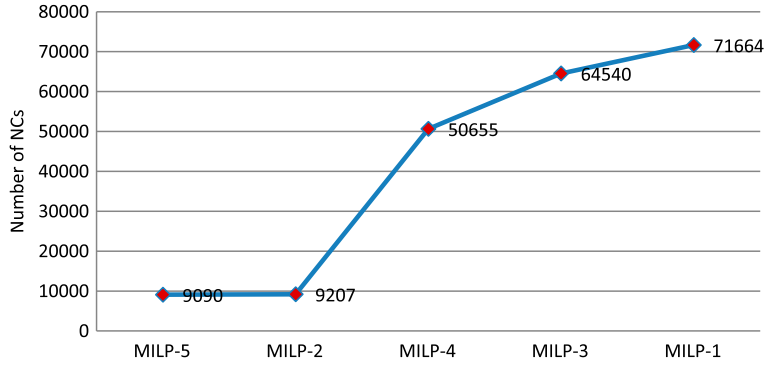


Figure 6. Performance evaluation of MILPs based on their NCs.

- *Size dimensionality*: Both MILP-1 and MILP-4 solved F-data to optimality up to SFJS10. Starting from MFJS1 and except for the last two instances of F-data (MFJS9 and MFJS10) both MILPs obtained feasible integer solutions. The quality of feasible integer solutions of MILP-4 is much better than those of the MILP-1.The MILP-2 obtained 14 optimal solutions on F-data as compared to MILP-3 and MILP-5 with 16 optimal solutions out of 20 instances. Therefore, in terms of size dimensionality, MILP-3 and MILP-5 are equally capable of solving different instances of F-JSSP with the same size dimensionality.
- *Quality of generated schedules*: The proposed position-based MILP-4 outperformed MILP-1 from literature on eight instances of F-data (MFJS1–MFJS8). Therefore, MILP-4 is superior to the MILP-1 in terms of quality of generated schedules. Both MILP-3 and MILP-5 solved two instances of F-data (MFJS4 and MFJS6) to optimality which had been solved to feasibility by MILP-2. All in all, MILP-3 and MILP-5 produced four new enhanced optimal and feasible solutions when comparing results obtained by MILP-2. MILP-5 improved the feasible solution obtained by MILP-3 on MFJS8. Overall, MILP-5 is the strongest MILP in terms of solution effectiveness. It is also concluded that *sequence-based MILPs* are superior to *position-based MILPs* generally speaking.

It goes without saying that the proposed position-based MILP-4 outperforms MILP-1 in all traditional and modern performance measures. Therefore, MILP-4 is the best thus far proposed position-based MILP. Among sequence-based MILP-s, MILP-5 completely outperformed MILP-2 from the literature. MILP-5 also produced one enhanced feasible solution on MFJS8.

To sum up, MILP-4 and MILP-5 developed and presented in this study are the best thus far proposed position-based and sequence-based MILPs, respectively. Among all the proposed MILPs, sequence-based models were superior to position-based models in terms of solution effectiveness and computational efficiency.

### 5.2 *Numerical investigations of meta-heuristics*

Having conducted numerical analyses between MILPs and known optimal, near optimal, and lower bounds of different instances of the F-data set, performance evaluations between state-of-the-art meta-heuristics have been done. The developed AISA algorithm was coded using Borland C++ programming language. Usually, as a common practice, in view of the non-deterministic nature of AISA, every instance of the problem is run for certain number of times and the best result is reported. In this study, except for the last three problem instances (MFJS8 to MFJS10) that were run for 5 times, the first 17 problem instances (SFJS1 to MFJS7) were run just once and AISA results were reported. The rationale behind not running the first 17 problem instances more than once was that they obtained better results vis-à-vis the best-performing meta-heuristic (AIA) even in the first run. As was mentioned in Section 2.2, for different purposes, six hybrid meta-heuristics by Fattahi, Mehrabad, and Jolai (2007) and one meta-heuristic (AIA) proposed by (Bagheri et al. 2010) have been selected to compare the developed algorithms against. There are several benchmarks in the literature, but the best data set is F-data set as it contains small- to medium-size instances of F-JSSP. This data set is suitable for this study, because the MILPs can be applied to it and their obtained results can be put as a basis for the comparison of meta-heuristics. Bagheri et al. (2010) applied AIA to this benchmark and showed that their algorithm outperformed other algorithms solving it. They applied their proposed AIA algorithm to BRdata, HUdata and Kacem benchmarks/data sets and showed that their AIA is superior to other available meta-heuristics in the literature solving these three benchmarks. However, with the experimental results performed and with the proposed AISA algorithm, it was shown that the developed hybridised meta-heuristic outperformed the AIA algorithm of Bagheri et al. (2010),

which has dominated the literature on a wide variety of benchmark problems. The RPD index is used (RPD = $\text{MILP}_{\text{best solution}} - \text{Algorithm}_{\text{solution}}/\text{MILP}_{\text{best solution}} \times 100\%$) to compare the performances of meta-heuristics. The RPD produced absolute percentage deviation (APD) values on the first 16 instances (up to MFJS6) of F-data as optimal solutions were available by the best MILP model. For the last four instances (MFJS6-MFJS10), the RPD was measured based on the best solutions of all algorithms.

Table 15 quantifies the improvements of the proposed AISA over previously proposed integrated meta-heuristics. AISA, AIA, iterated simulated annealing (ISA) and iterated Tabu Search (ITS) have produced RPDs of 0.056, 0.662, 0.8245, 12.25 and 14.51%, respectively. Here, two types of comparisons are conducted. First, the solutions obtained by AISA are compared with those of MILP-5 which is the best-performing MILP. In the first 16 instances of the F-data set, both MILP-5 and AISA found optimal solutions. For the next four instances (MFJS7–MFJS10), AISA produced better integer feasible solutions than MILP-5; note MILP-5 has not been solved for optimality for these larger size instances and was terminated after some pre-specified computational time (3600 s). The second best-performing meta-heuristic is AIA proposed by (Bagheri et al. 2010). AIA obtained fifteen best solutions vis-a-vis the AISA with 19 best solutions on all twenty instances of F-data set. The proposed AISA found better solutions than AIA on instances of MFJS2, MFJS3, MFJS5, MFJS6 and MFJS10, while the AIA only found one better solution than AISA on instance of MFJS8. MILP-5 obtained 16 optimal solutions on the first 16 instances of the problem. Therefore, AISA, MILP-5 and AIA are ranked first, second and third, respectively, in terms of obtaining optimal solutions. As for the quality of generated integer feasible solutions on the last four instances of F-data set (MFJS7–MFJS10), both the AISA and AIA outperformed MILP-5. Overall, AISA dominated AIA in terms of solution effectiveness. Among the integrated meta-heuristics proposed by (Fattahi, Mehrabad, and Jolai 2007) and Bagheri et al. (2010), AIA, ISA and ITS gained the RPD of 0.662, 12.25 and 14.51, respectively. By performing this analysis, it is ascertained that hierarchical approach (AISA) is stronger than the integrated approach. This superiority can be attributed to the fact that integrated approaches generated infeasible solutions, and they need to have a repair strategy to make infeasible solutions feasible.

Now that the superiority of the AISA has been established, it should be compared against hierarchical meta-heuristics proposed by Fattahi, Mehrabad, and Jolai (2007). Table 16 shows the comparative evaluation among the AISA and other hierarchical meta-heuristics in literature.

In Table 16, comparative performance evaluation is performed among the meta-heuristics following hierarchical approach. Four hierarchical hybrid meta-heuristics based on TS and SA has been proposed by Fattahi, Mehrabad, and

Table 15. Comparison with the state-of-the-art integrated approaches on F-data set.

| Instance | Size ($j$,$l$,$i$) | MILP-5 | | AISA | | | AIA | | ISA | | ITS | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | $C_{\max}$ | RPD% | $C_{\max}$ | RPD% | CPU (s) | $C_{\max}$ | RPD% | $C_{\max}$ | RPD% | $C_{\max}$ | RPD% |
| SFJS1 | 2.2.2 | **66**[a] | **0** | **66**[a] | 0 | 0.8 | **66**[a] | 0 | **66**[a] | **0** | **66**[a] | **0** |
| SFJS2 | 2.2.2 | **107**[a] | **0** | **107**[a] | 0 | 0.8 | **107**[a] | 0 | **107**[a] | **0** | **107**[a] | **0** |
| SFJS3 | 3.2.2 | **221**[a] | **0** | **221**[a] | 0 | 1.2 | **221**[a] | 0 | **221**[a] | **0** | **221**[a] | **0** |
| SFJS4 | 3.2.2 | **355**[a] | **0** | **355**[a] | 0 | 1.2 | **355**[a] | 0 | **355**[a] | **0** | 390[b] | 9.85 |
| SFJS5 | 3.2.2 | **119**[a] | **0** | **119**[a] | 0 | 1.2 | **119**[a] | 0 | **119**[a] | **0** | 137[b] | 15.1 |
| SFJS6 | 3.3.3 | **320**[a] | **0** | **320**[a] | 0 | 1.8 | **320**[a] | 0 | **320**[a] | **0** | **320**[a] | **0** |
| SFJS7 | 3.3.5 | **397**[a] | **0** | **397**[a] | 0 | 3 | **397**[a] | 0 | **397** | **0** | **397**[a] | **0** |
| SFJS8 | 3.3.4 | **253**[a] | **0** | **253**[a] | 0 | 2.4 | **253**[a] | 0 | **253**[a] | **0** | **253**[a] | **0** |
| SFJS9 | 3.3.3 | **210**[a] | **0** | **210**[a] | 0 | 1.8 | **210**[a] | 0 | **215**[a] | **0** | 215[b] | 2.38 |
| SFJS10 | 4.3.5 | **516**[a] | **0** | **516**[a] | 0 | 4 | **516**[a] | 0 | **516**[a] | **0** | 617[b] | 19.6 |
| MFJS1 | 5.3.6 | **468**[a] | **0** | **468**[a] | 0 | 6 | **468**[a] | 0 | 488[b] | 4.2 | 548[b] | 17.1 |
| MFJS2 | 5.3.7 | **446**[a] | **0** | **446**[a] | 0 | 7 | 448[b] | 0.4 | 478[b] | 7.2 | 457[b] | 2.5 |
| MFJS3 | 6.3.7 | **466**[a] | **0** | **466**[a] | 0 | 8.4 | 468[b] | 0.4 | 599[b] | 28.5 | 606[b] | 30 |
| MFJS4 | 7.3.7 | **554**[a] | **0** | **554**[a] | 0 | 9.8 | **554**[a] | **0** | 703[b] | 26.8 | 870[b] | 57 |
| MFJS5 | 7.3.7 | **514**[a] | **0** | **514**[a] | 0 | 9.8 | 527[b] | 2.5 | 674[a] | 31.1 | 729[b] | 41.8 |
| MFJS6 | 8.3.7 | **608**[a] | **0** | **608**[a] | 0 | 11.2 | 635[b] | 4 | 856[b] | 40.8 | 816[b] | 34.2 |
| MFJS7 | 8.4.7 | **881**[b] | **0.23** | **879**[b] | 0 | 11.2 | **879**[b] | **0** | 1066[b] | 21.3 | 1048[b] | 19.23 |
| MFJS8 | 9.4.8 | **894**[b] | **0** | **894**[b] | 1.13 | 14.4 | **884**[b] | **0** | 1328[b] | 50.23 | 1220[b] | 38 |
| MFJS9 | 11.4.8 | **1192**[b] | **9.56** | **1088**[b] | 0 | 17.6 | **1088**[b] | **0** | 1148[b] | 5.5 | 1124[b] | 3.31 |
| MFJS10 | 12.4.8 | **1276**[b] | **6.8** | **1196**[b] | 0 | 19.2 | 1267[b] | 5.94 | 1546[b] | 29.3 | 1737[b] | 12.35 |
| Average RPD | | | 0.8245 | | 0.056 | | | 0.662 | | 12.25 | | 14.51 |

Note: Boldfacing shows the best element in each row or best solution.
[a]Optimal solution.
[b]Best obtained so-far feasible solution.

Table 16. Comparison with the state-of-the-art hierarchical approaches on F-data set.

| Instance | Size ($j$,$l$,$i$) | MILP-5 $C_{max}$ | AISA | | HSA/SA | | HSA/TS | | HTS/TS | | HTS/SA | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | $C_{max}$ | RPD | $C_{max}$ | RPD | $C_{max}$ | RPD | $C_{max}$ | RPD | $C_{max}$ | RPD |
| SFJS1 | 2.2.2 | **66**[a] | **66**[a] | **0** | **66**[a] | **0** | **66**[a] | **0** | **66**[a] | **0** | **66**[a] | **0** |
| SFJS2 | 2.2.2 | **107**[a] | **107**[a] | **0** | **107**[a] | **0** | **107**[a] | **0** | **107**[a] | **0** | **107**[a] | **0** |
| SFJS3 | 3.2.2 | **221**[a] | **221**[a] | **0** | **221**[a] | **0** | **221**[a] | **0** | **221**[a] | **0** | **221**[a] | **0** |
| SFJS4 | 3.2.2 | **355**[a] | **355**[a] | **0** | **355**[a] | **0** | **355**[a] | **0** | **355**[a] | **0** | **355**[a] | **0** |
| SFJS5 | 3.2.2 | **119**[a] | **119**[a] | **0** | **119**[a] | **0** | **119**[a] | **0** | **119**[a] | **0** | **119**[a] | **0** |
| SFJS6 | 3.3.3 | **320**[a] | **320**[a] | **0** | **320**[a] | **0** | **320**[a] | **0** | **320**[a] | **0** | **320**[a] | **0** |
| SFJS7 | 3.3.5 | **397**[a] | **397**[a] | **0** | **397**[a] | **0** | **397**[a] | **0** | **397**[a] | **0** | **397**[a] | **0** |
| SFJS8 | 3.3.4 | **253**[a] | **253**[a] | **0** | **253**[a] | **0** | **253**[a] | **0** | **253**[a] | **0** | 256[a] | **0** |
| SFJS9 | 3.3.3 | **210**[a] | **210**[a] | **0** | **210**[a] | **0** | **210**[a] | **0** | **210**[a] | **0** | **210**[a] | **0** |
| SFJS10 | 4.3.5 | **516**[a] | **516**[a] | **0** | **516**[a] | **0** | **516**[a] | **0** | **516**[a] | **0** | 519[a] | 0.5 |
| MFJS1 | 5.3.6 | **468**[a] | **468**[a] | **0** | 479[b] | 2.3 | 491[a] | 4.9 | 469[a] | 0.2 | 469[a] | 4.9 |
| MFJS2 | 5.3.7 | **446** | **446**[a] | **0** | 495[b] | 11 | 482[a] | 8 | 482[a] | 8 | 468[a] | 4.9 |
| MFJS3 | 6.3.7 | **466**[a] | **466**[a] | **0** | 553[b] | 18.6 | 538[a] | 15.4 | 533[a] | 14.4 | 538[a] | 15.4 |
| MFJS4 | 7.3.7 | **554**[a] | **554**[a] | **0** | 656[b] | 18.4 | 650[a] | 17.3 | 634[a] | 14.4 | 618[a] | 11.6 |
| MFJS5 | 7.3.7 | **514**[a] | **514**[a] | **0** | 650[b] | 26.4 | 662[a] | 28.8 | 625[a] | 21.6 | 625[a] | 21.6 |
| MFJS6 | 8.3.7 | **608**[a] | **608**[a] | **0** | 762[b] | 25.3 | 785[a] | 29.1 | 717[a] | 17.9 | 730[a] | 20 |
| MFJS7 | 8.4.7 | 881[b] | 879[b] | **0** | 1020[b] | 16.04 | 1081[a] | 22.98 | 964[a] | 9.67 | 947[a] | 7.73 |
| MFJS8 | 9.4.8 | 894[b] | 894[b] | 0 | 1030[b] | 15.21 | 1122[a] | 22.5 | 970[a] | 8.5 | 922[a] | 3.13 |
| MFJS9 | 11.4.8 | 1192[LB] | 1088[b] | **0** | 1180[b] | 8.46 | 1243[a] | 14.25 | 1105[a] | 1.56 | 1105[a] | 1.56 |
| MFJS10 | 12.4.8 | 1276[LB] | 1196[b] | **0** | 1538[b] | 28.6 | 1615[a] | 35.03 | 1404[a] | 17.39 | 1384[a] | 15.71 |
| Total | | | | **0** | | 8.5155 | | 9.913 | | 5.681 | | 5.3515 |

Note: Boldfacing shows the best element in each row or best solution.
[a]Optimal solution.
[b]Best obtained so-far feasible solution.

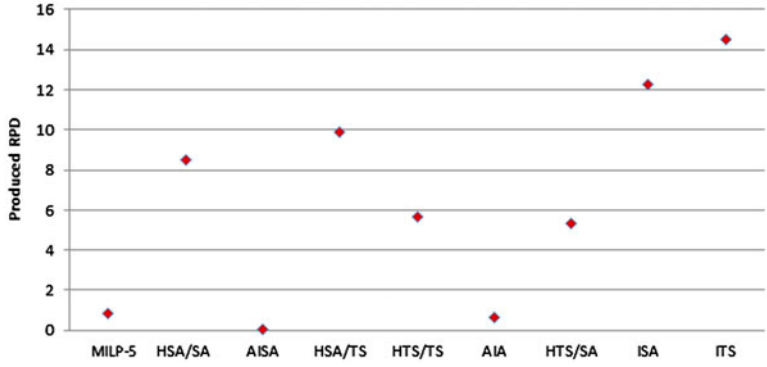


Figure 7. Comparative evaluation of meta-heuristics based on their produced RPDs.

Jolai (2007). The results obtained by AISA are used for comparison. AISA, HTS/SA, HSA/SA and HTS/TS obtained RPDs of 0, 5.3515, 5.681, 8.5155 and 9.913%, respectively. The best performing meta-heuristics could only compete with AISA on the first ten instances of the problem up to MJFS1. Other than the last four lower bound values of the proposed MILP, AISA obtained best solutions on all instances of the problem vs. other four hybrid meta-heuristics. Graphical representation of all the meta-heuristics based on their produced RPD is illustrated in Figure 7.

The obtained results acknowledge that AISA is highly efficient for F-JSSP and is more efficient than the recently proposed AIA. Another conclusion that can be made is that hierarchical approaches work better than integrated approaches. Having ensured that AISA is the best thus far proposed meta-heuristics; it is applied to the case study.

## 6. Case-study

Despite intensely growing competition in academic societies for developing state-of-the-art optimisation methodologies for wide variety of industrial applications including production scheduling environments, most industries are barely

familiar with these techniques and they use very basic techniques incorporated in commercial software packages to meet their short-term and long-term scheduling needs. In this study, the developed scheduling optimisation methodologies are applied to an industrial problem for demonstration and testing. The company is tier one supplier for many automotive companies in North America. In this study, the used case study is limited to the scheduling of mould making machines on the shop floor, while mould assembly operations are not being considered.

The mould and die manufacturing company makes different moulds used to create products such as tail lenses or head lamp reflectors for automotive manufacturers. They design and build prototype sand production thermo-plastic, thermoset, multi-colour and multi-material injection moulds using ejectable thermo plastic press and specialise in moulds for automobiles head and tail lights parts. A mould (Job) has two main parts: the core side and the cavity side (male/female). A typical mould consists of: (a) *Cavity*, (b) *Core*, (c) *Slides*, (d) *Retractor* and (e) *Clamp Plates*. The machining processes consist of the following operations: (1) *roughing*, (2) *semi-finishing*, (3) *stress relief*, (4) *finishing*, (5) *boring*, (6) *gun drilling*, (7) *carbon cutting for EDM processing*, (8) *electro discharge machining* (*EDM*) and finally (9) *mould assembly*. The scheduling of machining operations using 14 computer numerical control (CNC) machines is considered. All cavities and cores of a typical mould require all the above-mentioned operations. Slides need only four operations: roughing, finishing, carbon and EDM. Retractors require two operations: roughing and finishing, and finally, clamp plates need only a boring mill operation. Operations naming scheme is exemplified as follows: $O_{11}$ denotes the first operation of cavity#1 which is roughing. $O_{42}$ represents the second operation of part number 4 in retractor#1 which is finishing. Therefore, the first index in $O_{j,l}$, $j$ indicates the part number and the second index, $l$, denotes the required operation; in other words, each part is being considered a separate job for the sake of the demonstration. Table 17, shows the capabilities of each machine.

Table 17 demonstrates machine capabilities. Based on established policy in the mould and die company, certain priorities have been assigned to CNC machines to ensure the best quality and even distribution of workloads between machines. Table 18, shows the coefficients assigned to each machine for processing each operation. Table 18 reveals the fact that machines are flexible, but their flexibilities are not identical. In front of each operation, a number of capable machines has been cited. Therefore, this case study follows the principles of PF-JSSP as each machine performs certain operations and therefore different parts do not have total routing freedom. This means that for each part, certain preferred machines have been designated. The algorithm used (AISA) to solve this case study has to take care of machine assignment flexibility and should choose the best available machine.

Factors shown in Table 18 indicate how the different machine tools compared with each other in terms of processing time. The higher the factor assigned to a machine tool, the less desirable that machine becomes for processing certain

Table 17. List of machine capabilities.

| Operations | Names of eligible machines |
|---|---|
| Roughing | $M_1$ (AWEA) − $M_2$ (Johnford) − $M_3$ (Dynamic) − $M_4$ (Eumach) |
| Stress relief | $M_5$ (Outsourced) |
| Semi-finishing | $M_4$ (Eumach) − $M_3$ (Dynamic) |
| Finishing | $M_6$ (Exceeder 1) − $M_7$ (Exceeder 2) |
| Boring Mill | $M_8$ (Kuraki), $M_9$ (Parpas), $M_{10}$ (Takumi) |
| Gun-drilling | $M_{11}$ (Outsourced) |
| Carbon | $M_3$ (Dynamic) − $M_{12}$ (Datic) |
| E.D.M | $M_{13}$ (Techno) − $M_{14}$ (NX8) |

Table 18. Priority coefficient assigned to machines in shop floor.

| Operations | Names of eligible machines |
|---|---|
| Roughing | (1) $M_1$, (1.1) $M_2$, (1.2) $M_3$, (1.3) $M_4$ |
| Stress relief | (1) $M_5$ |
| Semi-finishing | (1) $M_4$, (1.1) $M_3$ |
| Finishing | (1) $M_6$, (1) $M_7$ |
| Boring Mill | (1) $M_8$, (1.1) $M_9$, (1.2) $M_{10}$ |
| Gun-drilling | (1) $M_{11}$ |
| Carbon | (1) $M_3$, (1.1) $M_{12}$ |
| E.D.M | (1) $M_{13}$, (1.1) $M_{14}$ |

Table 19. Operational requirements of different mould parts.

| No | No | Part names | Required operations |
|---|---|---|---|
| Mould # 1 | Part1 | Cavity1 | $O_{11}$, $O_{12}$, $O_{13}$, $O_{14}$, $O_{15}$, $O_{16}$, $O_{17}$, $O_{18}$ |
| | Part2 | Core1 | $O_{21}$, $O_{22}$, $O_{23}$, $O_{24}$, $O_{25}$, $O_{26}$, $O_{27}$, $O_{28}$ |
| | Part3 | Slide1 | $O_{31}$, $O_{32}$, $O_{33}$, $O_{34}$ |
| | Part4 | Retractor1 | $O_{41}$, $O_{42}$ |
| | Part5 | Clamp-Plates 1 | $O_{51}$ |
| Mould # 2 | Part6 | Cavity2 | $O_{61}$, $O_{62}$, $O_{63}$, $O_{64}$, $O_{65}$, $O_{66}$, $O_{67}$, $O_{68}$ |
| | Part7 | Core2 | $O_{71}$, $O_{72}$, $O_{73}$, $O_{74}$, $O_{75}$, $O_{76}$, $O_{77}$, $O_{78}$ |
| | Part8 | Slide2 | $O_{81}$, $O_{82}$, $O_{83}$, $O_{84}$ |
| | Part9 | Retractor2 | $O_{91}$, $O_{92}$ |
| | Part10 | Clamp-Plates 2 | $O_{101}$ |
| Mould # 3 | Part11 | Cavity3 | $O_{111}$, $O_{112}$, $O_{113}$, $O_{114}$, $O_{115}$, $O_{116}$, $O_{117}$, $O_{118}$ |
| | Part12 | Core3 | $O_{121}$, $O_{122}$, $O_{123}$, $O_{124}$, $O_{125}$, $O_{126}$, $O_{127}$, $O_{128}$ |
| | Part13 | Slide3 | $O_{131}$, $O_{132}$, $O_{133}$, $O_{134}$ |
| | Part14 | Retractor3 | $O_{141}$, $O_{142}$ |
| | Part15 | Clamp-Plates 3 | $O_{151}$ |
| Mould # 4 | Part16 | Cavity4 | $O_{161}$, $O_{162}$, $O_{163}$, $O_{164}$, $O_{165}$, $O_{166}$, $O_{167}$, $O_{168}$ |
| | Part17 | Core4 | $O_{171}$, $O_{172}$, $O_{173}$, $O_{1744}$, $O_{175}$, $O_{176}$, $O_{177}$, $O_{178}$ |
| | Part18 | Slide4 | $O_{181}$, $O_{182}$, $O_{183}$, $O_{184}$ |
| | Part19 | Retractor4 | $O_{191}$, $O_{192}$ |
| | Part20 | Clamp-Plates 4 | $O_{201}$ |


Figure 8. Decoded solution of the case study in format of a Gantt chart.

operations. The core of this study is the proper assignment and sequencing of four moulds (jobs) consisting of 23 parts with 92 operations on 14 machines.

Having done rigorous assessment of the performance of the proposed AISA through comparative analyses, the developed AISA has been confidently applied to solve real case study of PF-JSSP with 23 parts (92 operations) on 14 flexible machining centres. The AISA utilised runtime of 56 s ($n*m*0.2$ s) to solve the case study, where $n=20-$number of parts and $m=14-$number of CNC machines. As the results show, the developed AISA is a rela-

Table 20. Decoded solution of AISA with starting and completing time of each operation.

| | |
|---|---|
| $M_1$ | $O_{11}$(0–60), $O_{121}$(60–132), $O_{161}$(132–187), $O_{31}$(187–198), $O_{171}$(198–265), $O_{111}$(265–325), $O_{141}$(325–339), $O_{91}$(336–371) |
| $M_2$ | $O_{21}$(0–79), $O_{71}$(79–153), $O_{131}$(153–169), $O_{61}$(169–231) |
| $M_3$ | $O_{16}$(261–345), $O_{25}$(345–429), $O_{76}$(84–513), $O_{176}$(513–549), $O_{166}$(549–585), $O_{126}$(585–669), $O_{66}$(669–753), $O_{116}$(753–837) |
| $M_4$ | $O_{13}$(145–169), $O_{23}$(169–193), $O_{73}$(233–281), $O_{163}$(281–317), $O_{173}$(335–371), $O_{123}$(371–461), $O_{113}$(461–497), $O_{63}$(497–545) |
| $M_5$ | $O_{12}$(60–145), $O_{191}$(145–168), $O_{162}$(187–263), $O_{172}$(265–335), $O_{41}$(-351), $O_{122}$(351–425) |
| $M_6$ | $O_{81}$(0–30), $O_{181}$(30–53), $O_{22}$(79–159), $O_{72}$(159–233), $O_{62}$(233–318), $O_{112}$(325–419), $O_{183}$(419–439), $O_{83}$(432–439), $O_{177}$(549–575), $O_{167}$(585–617), $O_{17}$(617–649), $O_{67}$(753–774), $O_{117}$(837–858) |
| $M_7$ | $O_{132}$(169–189), $O_{182}$(189–210), $O_{32}$(210–245), $O_{164}$(317–371), $O_{174}$(371–431), $O_{92}$(431–467), $O_{192}$(-505), $O_{64}$(545–597) |
| $M_8$ | $O_{82}$(30–50), $O_{14}$(169–221), $O_{24}$(221–291), $O_{74}$(291–351), $O_{42}$(351–391), $O_{124}$(461–521), $O_{114}$(521–573), $O_{142}$(573–589) |
| $M_9$ | $O_{51}$(0–36), $O_{15}$(221–261), $O_{115}$(573–613), $O_{65}$(597–642) |
| $M_{10}$ | $O_{101}$(0–45), $O_{165}$(371–425), $O_{151}$(425–470), $O_{125}$(521–580) |
| $M_{11}$ | $O_{75}$(351–416), $O_{175}$(431–504), $O_{201}$(504–554), $O_{26}$ (554–638), |
| $M_{12}$ | $O_{133}$(189–201), $O_{33}$(245–270), $O_{77}$(513–553), $O_{27}$(638–685), $O_{127}$(685–715) |
| $M_{13}$ | $O_{178}$(575–621), $O_{84}$(621–641), $O_{18}$(649–675), $O_{34}$(675–694), $O_{68}$(774–800), $O_{118}$(858–881) |
| $M_{14}$ | $O_{134}$(201–236), $O_{78}$(553–613), $O_{184}$(613–652), $O_{128}$(715–769), $O_{168}$(769–795), $O_{28}$(795–855) |

tively fast algorithm capable of finding near-optimal solutions for any large size problem. In order to solve the case-study, data regarding both the processing times of all parts on different machines and also their eligible machines were collected. Table 19 demonstrates the operational requirements and coding of the different parts of the mould.

Gantt chart for solution obtained from the AISA after decoding is shown in Figure 8. The Gantt chart consists of starting times and completion times of all operations. For example, $O_{13}$ on $M_4$ has processing time of 24 and completion time is 169 which is the summation of processing times of preceding operations of $O_{13}$($O_{11}=60$, $O_{12}=80$ and $O_{13}=24$). The company used to complete these 92 operations on its current resources in around 960 h which is equivalent to 120 working shifts (two shifts per day including weekdays and weekends). The AISA obtained the make-span of 881 h for manufacturing of 92 operations belonging to 4 different moulds. The difference between the old company schedule and the currently obtained solution is 79 h which is equivalent to nearly 10 working shifts of total savings. Considering this production time saving, the company reduction in manufacturing time is 8.3%, which translates into savings of wages and costs. The decoded solution of AISA for the case study is shown in Table 20 and Figure 8. In Figure 8, each part is shown with a unique colour so that the processing route of each part on different machines can be easily tracked. Each operation of a part has two numbers; the first is operation processing time, and the second is the summation of processing times a part has received so far.

## 7. Conclusion and future research

In this study, an industrial scheduling problem in the form of partially flexible job shops was tackled. To this end, novel solution methodologies were proposed and utilised. Two new position-based MILP models were developed for the detailed characterisation of a typical industrial partially flexible jobs shop scheduling. Parametric size complexities of the proposed models under partial and total cases of F-JSSP were investigated. Then, based on efficiency of their size complexity, one of the models was selected for numerical comparison purposes. The developed MILP possesses 59.5, 77.6 and 47.5% fewer numbers of binary integer and non-negative decision variables and also constraints, respectively. The combined effect of such improvements led to both computational efficiency of 1026% (RPD of 1026%) and the introduction of six new integer feasible solutions for the F-data set which used to be lower bound values. This model can be used for solving small- to medium-sized problems with the size complexity of eight jobs on seven machines $(8!)^7$as opposed to previous mathematical models that could only solve up to four jobs and on five machines $(4!)^5$. Therefore, the proposed MILP is capable of solving the F-JSSP to larger instances of the problem.

MILPs have proven their efficiencies in few shop types with relatively less computational complexity such as single machines, and flow shop. It is notoriously recognised that MILPs are confined to small- to medium-size instances for F-JSSPs as they present tremendous size complexity of $(n!)^m$. Therefore, to overcome this deficiency and acquire solutions for the real industrial problem, a new meta-heuristic algorithm was developed by hybridisation of Artificial Immune Algorithm and Simulated Annealing. To ensure the suitability of the proposed hybrid algorithm, it was compared with MILP and seven other state-of-the-art meta-heuristics on a set of standard benchmark problems (F-data set). AISA obtained 14 solutions similar to those of AIA algorithm from literature. AISA found five new best solutions in a benchmark of 20 instances for the make-span optimisation criterion.AIA outperformed the AISA in only one of the instances of the problem (excluding the obtained lower bound values of the MILP). Otherwise, this superiority of AISA is attributed to the strength of the SA in finding the schedules with lower make-span in hyper-mutation phase. Once the

effectiveness and suitability of the proposed meta-heuristic was ascertained to tackle the case study, the hybrid algorithm was applied to data set collected from mould- and die-making shop. The proposed hybrid algorithm, obtained lower production times as opposed to their real production time and proved its suitability for solving real case study of 20 parts (92 operations) on 14 flexible machining centres that is typical size of many real industrial scheduling settings. All in all, the proposed AISA improved the company's scheduling efficiency by 8.3%.

To summarise, this study is seen to contribute theoretically and practically to the solution and modelling of production scheduling in general and flexible job shops in particular. For extension of work, mathematical programming algorithms, multi-objective, dynamic and distributed versions of F-JSSP can be explored as different avenues for future research.

**Acknowledgements**

This research was conducted in the Intelligent Manufacturing Systems Center at the University of Windsor, Canada. Research funding from the Canada Research Chairs program and the Natural Sciences and Engineering Council (NSERC) of Canada are gratefully acknowledged.

## References

Al-Hinai, N., and T. Y. ElMekkawy. 2011. "An Efficient Hybridized Genetic Algorithm Architecture for the Flexible Job Shop Scheduling Problem." *Flexible Services and Manufacturing Journal* 23 (1): 64–85.

Bagheri, A., M. Zandieh, I. Mahdavi, and M. Yazdani. 2010. "An Artificial Immune Algorithm for the Flexible Job-Shop Scheduling Problem." *Future Generation Computer Systems* 26 (4): 533–541.

Barnes, J. W., and J. B. Chambers. 1996. "Flexible Job Shop Scheduling by Tabu Search." Graduate program in operations research and industrial engineering. Technical Report ORP 9609, University of Texas, Austin. http://www.cs.utexas.edu/users/jbc/

Bowman, E. H. 1959. "Schedule-Sequencing Problem." *Operations Research* 7 (5): 612–614.

Brandimarte, P. 1993. "Routing and Scheduling in a Flexible Job Shop by Tabu Search." *Annals of Operations Research* 41 (1–4): 157–183.

Brucker, P., and R. Schlie. 1990. "Job-Shop Scheduling with Multi-Purpose Machines." *Computing* 45 (4): 369–375.

Chen, H., J. Ihlow, and C. Lehmann. 1999. "A Genetic Algorithm for Flexible Job-Shop Scheduling." *Proceedings of International Conference on Robotics and Automation*, 10–15 May 1999, Piscataway, NJ: IEEE.

Dauzere-Peres, S., and J. Paulli. 1997. "An Integrated Approach for Modeling and Solving the General Multiprocessor Job-Shop Scheduling Problem Using Tabu Search." *Annals of Operations Research* 70 : 281–306.

Fattahi, P., M. S. Mehrabad, and F. Jolai. 2007. "Mathematical Modeling and Heuristic Approaches to Flexible Job Shop Scheduling Problems." *Journal of Intelligent Manufacturing* 18 (3): 331–342.

French, S. 1982. *Sequencing and Scheduling: An Introduction to the Mathematics of the Job-Shop*. Chichester: Ellis Horwood.

Gao, J., L. Sun, and M. Gen. 2008. "A Hybrid Genetic and Variable Neighborhood Descent Algorithm for Flexible Job Shop Scheduling Problems." *Computers and Operations Research* 35 (9): 2892–2907.

Garey, M. R., D. S. Johnson, and R. Sethi. 1976. "The Complexity of Flowshop and Jobshop Scheduling." *Mathematics of Operations Research* 1 (2): 117–129.

Goldberg, D. E. 1989. *Genetic Algorithms in Search, Optimization and Machine learning*. Reading, MA: Addison-Wesley.

Hurink, J., B. Jurisch, and M. Thole. 1994. "Tabu Search for the Job-Shop Scheduling Problem with Multi-Purpose Machines." *OR Spektrum* 15 (4): 205–215.

Kacem, I., S. Hammadi, and P. Borne. 2002. "Approach by Localization and Multiobjective Evolutionary Optimization for Flexible Job-Shop Scheduling Problems." IEEE Transactions on Systems, Man and Cybernetics, Part C (Applications and Reviews) 32 (1): 1–13.

Kesen, S. E., and Z. Gungor. 2012. "Job Scheduling in Virtual Manufacturing Cells with Lot-Streaming Strategy: A New Mathematical Model Formulation and a Genetic Algorithm Approach." *Journal of the Operational Research Society* 63 (5): 683–695.

Liao, C.-J., and C.-T. You. 1992. "Improved Formulation for the Job-Shop Scheduling Problem." *Journal of the Operational Research Society* 43 (11): 1047–1054.

Manne, A. S. 1960. "On Job-Shop Scheduling Problem." *Operations Research* 8 (2): 219–223.

Mastrolilli, M., and L. M. Gambardella. 2000. "Effective Neighbourhood Functions for the Flexible Job Shop Problem." *Journal of Scheduling* 3 (1): 3–20.

Naderi, B., and R. Ruiz. 2010. "The Distributed Permutation Flowshop Scheduling Problem." *Computers and Operations Research* 37 (4): 754–768.

Naderi, B., M. Zandieh, A. K. Ghoshe Balagh, and V. Roshanaei. 2009. "An Improved Simulated Annealing for Hybrid Flowshops with Sequence-Dependent Setup and Transportation Times to Minimize Total Completion Time and Total Tardiness." *Expert Systems with Applications* 36 (6): 9625–9633.

Naderi, B., M. Zandieh, and V. Roshanaei. 2009. "Scheduling Hybrid Flowshops with Sequence Dependent Setup Times to Minimize Make-Span and Maximum Tardiness". *International Journal of Advanced Manufacturing Technology* 41 (11): 1186–1198.

Ozguven, C., L. Ozbakir, and Y. Yavuz. 2010. "Mathematical Models for Job-Shop Scheduling Problems with Routing and Process Plan Flexibility." *Applied Mathematical Modelling* 34 (6): 1539–1548.

Ozguven, C., Y. Yavuz, and L. Ozbakir. 2012. "Mixed Integer Goal Programming Models for the Flexible Job-Shop Scheduling Problems with Separable and Non-Separable Sequence Dependent Setup times." *Applied Mathematical Modelling* 36 (2): 846–858.

Pan, C.-H. 1997. "Study of Integer Programming Formulations for Scheduling Problems." *International Journal of Systems Science* 28 (1): 33–41.

Pezzella, F., G. Morganti, and G. Ciaschetti. 2008. "A Genetic Algorithm for the Flexible Job-Shop Scheduling Problem." *Computers and Operations Research* 35 (10): 3202–3212.

Pinedo, M. 2002. *Scheduling: Theory, Algorithms and Systems*. Englewood Cliffs, NJ: Prentice-Hall.

Roshanaei, V., H. ElMaraghy, and A. Azab. 2012. "Sequence-based MILP Modeling for Flexible Job Shop Scheduling." Proceedings of IIE conference & expo 2012, Orlando, Florida, USA, May 19–23.

Roshanaei, V., M. M. Seyyed Esfehani, and M. Zandieh. 2010. "Integrating Non-Preemptive Open Shops Scheduling with Sequence-Dependent Setup times Using Advanced Metaheuristics." *Expert Systems with Applications* 37 (1): 259–266.

Stafford, E. F. 1988. "On the Development of a Mixed-Integer Linear Programming Model for the Flowshop Sequencing Problem." *Journal of the Operational Research Society* 39 (12): 1163–1174.

Stafford, E. F., Jr., and F. T. Tseng. 2005. "Comparative Evaluation of MILP Flowshop Models." *Journal of the Operational Research Society* 56 (1): 88–101.

Vancza, J., T. Kis, and A. Kovacs. 2004. "Aggregation – The Key to Integrating Production Planning and Scheduling." *CIRP Annals – Manufacturing Technology* 53 (1): 377–380.

Wagner, H. M. 1959. "An Integer Linear-Programming Model for Machine Scheduling." *Nav. Res. Logist. Quart.* 6 : 131–140.

Wiendahl, H. P., H. A. ElMaraghy, P. Nyhuis, M. F. Zah, H. H. Wiendahl, N. Duffie, and M. Brieke. 2007. "Changeable Manufacturing – Classification, Design and Operation." *CIRP Annals – Manufacturing Technology* 56 (2): 783–809.

Wilson, J. M. 1989. "Alternative Formulations of a Flow-Shop Scheduling Problem." *Journal of the Operational Research Society* 40 (4): 395–399.

Xia, W., and Z. Wu. 2005. "An Effective Hybrid Optimization Approach for Multi-Objective Flexible Job-Shop Scheduling Problems." *Computers and Industrial Engineering* 48 (2): 409–425.

Yazdani, M., M. Amiri, and M. Zandieh. 2010. "Flexible Job-Shop Scheduling with Parallel Variable Neighborhood Search Algorithm." *Expert Systems with Applications* 37 (1): 678–687.

Zandieh, M., S. M. T. Fatemi Ghomi, and S. M. Moattar Husseini. 2006. "An Immune Algorithm Approach to Hybrid Flow Shops Scheduling with Sequence-Dependent Setup times." *Applied Mathematics and Computation (New York)* 180 (1): 111–127.

Zhang, G., L. Gao, and Y. Shi. 2011. "An Effective Genetic Algorithm for the Flexible Job-Shop Scheduling Problem." *Expert Systems with Applications* 38 (4): 3563–3573.